BULL DOG SOLUTION

Paquete de documentacion del producto

Sistema de Gestion de Ordenes de Servicio

Contenido: producto, requisitos, alcance, roadmap, operacion y anexos tecnicos.

Version de compilacion: 1.0

Fecha: 3 de septiembre de 2026

Nota: Los diagramas Mermaid fueron renderizados como graficos vectoriales.

Contenido del paquete

1. **1. Resumen general del producto**
   README.md
2. **2. Especificacion de Requisitos de Software (ERS)**
   docs/ERS.md
3. **3. Especificacion tecnica del MVP**
   SPECS.md
4. **4. Ruta de trabajo y avance**
   ROADMAP.md
5. **5. Matriz de trazabilidad**
   docs/traceability.md
6. **6. Plan de piloto y migracion desde AppSheet**
   docs/appsheet-migration-plan.md
7. **7. Decisiones temporales de desarrollo**
   docs/development-decisions.md
8. **8. Variables, secretos y responsables**
   docs/environment-ownership.md
9. **9. Seguridad, calidad y observabilidad**
   docs/security-and-quality.md
10. **10. Despliegue y ambiente de staging**
    docs/staging-and-deployment.md
11. **11. Especificacion del reporte de orden de servicio**
    docs/service-order-report-spec.md
12. **12. Indice de documentacion tecnica**
    docs/technical/README.md
13. **13. Arquitectura**
    docs/technical/architecture.md
14. **14. Modelo de datos**
    docs/technical/data-model.md
15. **15. Backend y flujos**
    docs/technical/backend-and-flows.md
16. **16. API HTTP**
    docs/technical/api.md
17. **17. Frontend**
    docs/technical/frontend.md
18. **18. Infraestructura y operacion**
    docs/technical/infrastructure-and-operations.md
19. **19. Seguridad y calidad tecnica**
    docs/technical/security-and-quality.md

Bull Dog Solution — Órdenes de servicio

Sistema web responsivo para preparar, asignar, ejecutar y auditar órdenes de instalación, revisión y desinstalación de equipos en unidades vehiculares.

> Este README describe el estado objetivo del producto una vez concluido el MVP. La implementación se encuentra organizada en ROADMAP.md y su definición técnica completa está en SPECS.md.

La documentación del estado técnico implementado, incluidos los diagramas de arquitectura y entidad-relación, está disponible en docs/technical/README.md.

Descripción

La plataforma sustituye el flujo operativo realizado en AppSheet y reúne en un solo lugar el trabajo administrativo y de campo:

- Los administradores crean órdenes, seleccionan clientes, registran unidades, asignan equipos y eligen al técnico responsable.
- Los técnicos consultan sus servicios, verifican equipos, documentan cada unidad con checklist y fotografías, reportan discrepancias y obtienen la firma del cliente.
- Google Sheets conserva la autoridad sobre los catálogos de clientes y equipos.
- PostgreSQL mantiene los datos transaccionales, el historial y el estado de sincronización.
- Las fotografías y firmas se almacenan de forma privada en un servicio compatible con Amazon S3.

Funcionalidades principales

Administración

- Autenticación segura y administración local de usuarios y técnicos.
- Creación de órdenes con varias unidades y varios equipos por unidad.
- Consulta de clientes y equipos directamente desde Google Sheets.
- Asignación y reasignación de técnicos.
- Seguimiento del estado y progreso de cada servicio.
- Detección visible de conflictos cuando una fila de Sheets cambia externamente.
- Consulta de evidencias, firma, discrepancias, auditoría y sincronización.
- Reporte PDF de cada orden completada con portada, detalle por unidad/equipo y fotografías en formato amplio.
- Reintento controlado de operaciones fallidas con Google Sheets.

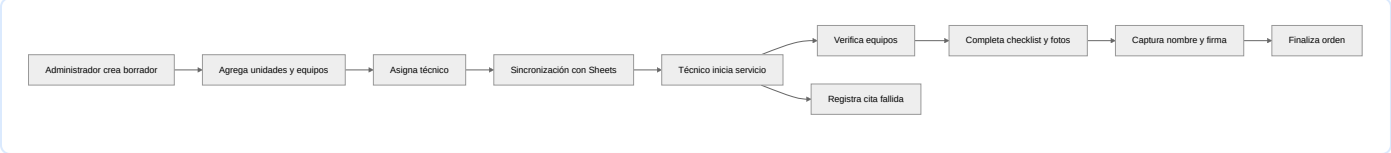
Operación técnica

- Vista mobile-first de las órdenes asignadas.
- Verificación individual de equipos.
- Reporte de diferencias de ID, IMEI, serie o unidad sin bloquear el trabajo.
- Checklist posterior al servicio para tablero, cristales, accesorios, molduras y carrocería.
- Estados `Bien`, `Mal` y `No aplica`, con observación obligatoria para elementos en mal estado.
- Captura de al menos una fotografía por categoría.
- Firma única del cliente y registro del nombre de la persona firmante.
- Cierre normal con validación integral o cierre simplificado como cita fallida.

Seguridad y trazabilidad

- Access y refresh tokens JWT con rotación y revocación de sesiones.
- Autorización por rol y pertenencia de la orden.
- Validación de todas las entradas mediante Valibot.
- Almacenamiento privado y URLs prefirmadas de corta duración.
- Auditoría de operaciones administrativas y cambios relevantes.
- Idempotencia, concurrencia optimista y outbox para efectos externos.
- Logs estructurados con redacción de secretos y datos sensibles.

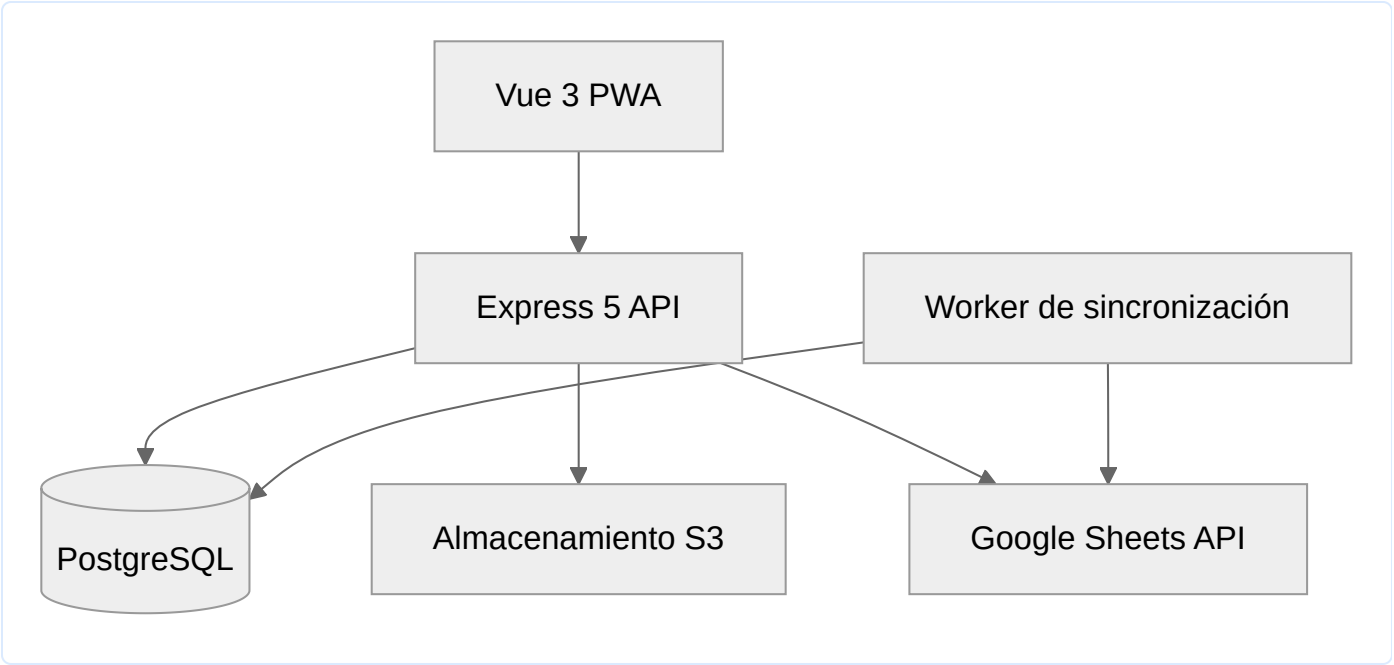
Flujo del servicio



Una discrepancia puede registrarse durante la verificación y no impide finalizar la orden cuando los demás requisitos están completos.

Arquitectura

El proyecto utiliza un monolito modular en un monorepo. El frontend y la API se desarrollan y construyen por separado, mientras que las reglas de negocio permanecen en el backend.



Capa	Tecnologías
Lenguaje	TypeScript
Frontend	Vue 3, Vite, Vue Router, Pinia, TanStack Query
Interfaz	Naive UI, Tailwind CSS, VueUse
API	Node.js, Express 5, Valibot
Persistencia	PostgreSQL, Prisma
Autenticación	JWT, Argon2id
Integraciones	Google Sheets API, API compatible con Amazon S3
Documentación	OpenAPI 3.1, Swagger UI
Pruebas	Vitest, Supertest, Playwright
Infraestructura	Docker, Docker Compose, pnpm

Estructura del repositorio

```
.
├── apps/
│   ├── api/                # API Express y módulos de negocio
│   └── web/                # Aplicación Vue y PWA
├── packages/
│   ├── contracts/          # DTOs y esquemas Valibot compartidos
│   ├── eslint-config/      # Configuración común de lint
│   └── typescript-config/  # Configuración común de TypeScript
├── prisma/
│   ├── migrations/         # Historial de migraciones
│   ├── schema.prisma       # Modelo de datos
│   └── seed.ts             # Datos iniciales de desarrollo
├── infra/
│   └── docker/             # Archivos de infraestructura local
├── docker-compose.yml
├── pnpm-workspace.yaml
├── ROADMAP.md
└── SPECS.md
```

Los módulos siguen slicing vertical. Cada dominio contiene sus rutas, controladores, servicios, persistencia, contratos y pruebas. Las integraciones con Google Sheets y S3 se consumen a través de interfaces reemplazables.

Estados de una orden

Una orden normal sigue este recorrido:

```
DRAFT → ASSIGNED → IN_PROGRESS → COMPLETED
                 → REQUIRES_CORRECTION → IN_PROGRESS
       → CANCELLED
```

Una cita fallida utiliza el flujo:

```
ASSIGNED → IN_PROGRESS → FAILED_APPOINTMENT
```

Para completar un servicio normal, todas las unidades deben tener sus equipos verificados, checklist completo y fotografías requeridas. La orden también debe incluir el nombre y la firma del cliente. Una cita fallida requiere únicamente un motivo.

Integración con Google Sheets

Google Sheets permanece como fuente oficial de los catálogos externos:

- `Clientes`: lectura de `IDCliente` y `Cliente`.
- `SALIDAS`: lectura del catálogo de equipos y actualización limitada de su asignación.

La aplicación solo puede escribir las columnas autorizadas:

Columna	Contenido
`J`	Cliente
`L`	Unidad
`M`	Marca/modelo de la unidad
`N`	Año
`O`	Número de serie
`P`	Tipo de servicio, cuando el negocio lo habilite

Las columnas `A`, `I` y `K` nunca son modificadas. Antes de cada escritura, la API vuelve a localizar el equipo por ID e IMEI, compara el snapshot conocido y detiene la operación con un conflicto HTTP `409` si detecta cambios externos.

Requisitos

- Node.js 24.20.0 LTS, fijado en `.nvmrc` y `.node-version`.
- pnpm 11.9.0 mediante Corepack.
- Docker y Docker Compose.
- Documento de Google Sheets de desarrollo compartido con una cuenta de servicio.
- Credenciales de Google Sheets y almacenamiento S3 para entornos no locales.

Puesta en marcha local

1. Activar la versión fijada de pnpm e instalar las dependencias:

```
corepack enable
pnpm install --frozen-lockfile
```

2. Crear la configuración local a partir del archivo de ejemplo:

```
cp .env.example .env
```

3. Para trabajar con recarga en caliente, levantar PostgreSQL y MinIO:

```
docker compose up -d postgres minio minio-init
```

4. Aplicar las migraciones y cargar datos iniciales:

```
pnpm prisma:migrate
pnpm prisma:seed
```

El seed local crea `admin@example.invalid` y `technician@example.invalid`. Cambia `SEED_ADMIN_PASSWORD` antes de ejecutarlo fuera de un entorno local; en producción es obligatorio.

5. Iniciar API y web:

```
pnpm dev
```

La aplicación web queda disponible por defecto en `http://localhost:5173` y la API en `http://localhost:3000`. `VITE_DEV_PORT`, `VITE_API_PROXY_TARGET` y `PORT` permiten usar puertos diferentes sin modificar código.

Como alternativa, el stack completo compilado se levanta con:

```
docker compose up --build -d
```

En esta modalidad, la web se publica por defecto en `http://localhost:4173`, la API en `http://localhost:3000` y la consola de MinIO en `http://localhost:9001`. Los puertos pueden sobrescribirse en `.env` con las variables documentadas en `.env.example`.

Configuración

La aplicación valida su configuración al iniciar y se detiene si falta una variable obligatoria. Los grupos principales son:

Grupo	Variables principales
Aplicación	`NODE_ENV`, `PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL`
JWT	Claves pública/privada, issuer, audience y TTL de cada token
Cookies	Dominio, modo seguro y política SameSite
Objetos	Endpoint, región, bucket y credenciales S3
Google Sheets	ID del documento, cuenta de servicio y nombres de pestañas
Límites	Tamaños de evidencia/firma, TTL de URLs y caché de catálogos
Resiliencia	Timeouts, reintentos, circuit breaker y frecuencia del outbox
Logs	`LOG_LEVEL`

Los secretos no deben guardarse en el repositorio. En producción se utilizan identidades administradas o un gestor de secretos cuando el proveedor lo permita.

Comandos disponibles

Comando	Descripción
`pnpm dev`	Inicia API y web en desarrollo
`pnpm build`	Construye todas las aplicaciones y paquetes
`pnpm format`	Formatea los archivos del repositorio
`pnpm format:check`	Verifica el formato sin escribir cambios
`pnpm lint`	Ejecuta el análisis estático
`pnpm typecheck`	Comprueba TypeScript estricto
`pnpm test`	Ejecuta pruebas unitarias y de integración
`pnpm test:e2e`	Ejecuta escenarios Playwright
`pnpm docs:build`	Genera OpenAPI 3.1
`pnpm docs:validate`	Valida el contrato de la API
`pnpm prisma:migrate`	Aplica migraciones de base de datos
`pnpm prisma:seed`	Carga datos iniciales de desarrollo
`pnpm prisma:validate`	Valida la configuración y el esquema Prisma

API y documentación

La API utiliza el prefijo `/api/v1` y responde en JSON con un `requestId` para trazabilidad.

- Especificación OpenAPI: `/api/v1/openapi.json`.
- Swagger UI: `/api/docs`.
- Estado de proceso: `/health/live`.
- Estado de dependencias: `/health/ready`.

Los endpoints abarcan autenticación, usuarios, técnicos, catálogos externos, órdenes, unidades, equipos, checklist, evidencias, firma, auditoría y sincronización.

Pruebas

La estrategia combina:

- Pruebas unitarias para validaciones, transiciones, autorización, rotación de tokens e identificación de equipos.
- Pruebas de integración para Express, PostgreSQL, almacenamiento de objetos, Sheets simulado, outbox e idempotencia.

- Pruebas de contrato para comparar OpenAPI con respuestas reales de la API.
- Pruebas end-to-end de los recorridos completos de administrador y técnico.

Los escenarios automatizados nunca escriben en la hoja de producción.

Despliegue

El producto se despliega con entornos separados de desarrollo, staging y producción:

- Frontend estático servido mediante CDN o reverse proxy.
- API ejecutada como contenedor Node.js.
- PostgreSQL administrado con respaldos automáticos y restauración probada.
- Bucket S3 privado con bloqueo de acceso público.
- Reverse proxy con TLS y, preferentemente, API bajo el mismo dominio público.
- Google Sheets distinto para staging y producción.

El pipeline ejecuta instalación reproducible, lint, typecheck, pruebas, generación de OpenAPI, builds, escaneos, migraciones controladas, despliegue y smoke tests.

Alcance del MVP

El MVP no incluye operación offline completa, aplicaciones móviles nativas, edición de los catálogos externos, facturación, cuentas bancarias, tarifas de técnicos ni microservicios. Tampoco modifica automáticamente `Fecha de salida`, `Estado` o `Estado del servicio` en Google Sheets.

Documentación del proyecto

- Especificación técnica
- Ruta de implementación

Especificación de Requisitos de Software (ERS)

Sistema de Gestión de Órdenes de Servicio — Bull Dog Solution

---

Control del Documento

- **Proyecto:** Bull Dog Solution — Órdenes de servicio
- **Versión:** 1.0.0
- **Fecha:** 2026-09-02
- **Autor / Organización:** Bull Dog Solution
- **Estado:** Propuesta de Especificación Formal (ERS / IEEE 830 / ISO/IEC/IEEE 29148)

---

1. Introducción

1.1 Propósito

El propósito de este documento es especificar formalmente los requisitos funcionales y no funcionales para el desarrollo del **Sistema de Gestión de Órdenes de Servicio de Bull Dog Solution**. Este documento sirve como contrato técnico y funcional entre los interesados de negocio, administradores de operaciones, técnicos en campo y el equipo de ingeniería de software.

1.2 Alcance del Producto

El sistema es una plataforma web responsiva (PWA básica) orientada a sustituir la operativa manual basada en AppSheet. Centraliza la preparación, asignación, ejecución técnica en campo, verificación de equipamiento telemático, checklist vehicular, captura de evidencia multimedia, firmas de conformidad, auditoría de eventos y sincronización bidireccional controlada con Google Sheets.

Objetivos clave:

1. Eliminar inconsistencias en la captura de datos en campo.
2. Garantizar la trazabilidad del estado físico de las unidades antes y después del servicio.
3. Mantener a Google Sheets como catálogo maestro de clientes y almacén de inventario (`SALIDAS`), sin comprometer la integridad transaccional del sistema.
4. Proveer reportes operativos, técnicos y gerenciales en tiempo real y formatos descargables (PDF, Excel, CSV).

1.3 Definiciones, Acrónimos y Abreviaturas

- **ERS:** Especificación de Requisitos de Software.

- **PWA:** Progressive Web Application.
- **IMEI:** International Mobile Station Equipment Identity.
- **S3:** Simple Storage Service (o API compatible como MinIO/Cloudflare R2).
- **JWT:** JSON Web Token.
- **Outbox Pattern:** Patrón arquitectónico para publicación garantizada y asíncrona de eventos transaccionales.
- **Checklist Post-Servicio:** Inspección del estado de la unidad vehicular realizada al concluir los trabajos técnicos.
- **Discrepancia:** Diferencia detectada en campo entre el equipo físico asignado (IMEI, serie, ID, unidad) y lo registrado en la orden.

1.4 Referencias

- Documento de Especificaciones Técnicas del Proyecto: `SPECS.md`
- Hoja de Ruta de Implementación: `ROADMAP.md`
- Estándar IEEE 830-1998 / ISO/IEC/IEEE 29148:2018 para Requisitos de Software.

1.5 Resumen del Documento

El resto del documento detalla la perspectiva del producto, actores, casos de uso, requisitos funcionales (incluyendo un módulo integral de reportes y analítica), requisitos no funcionales, interfaces de usuario y externas, y restricciones tecnológicas.

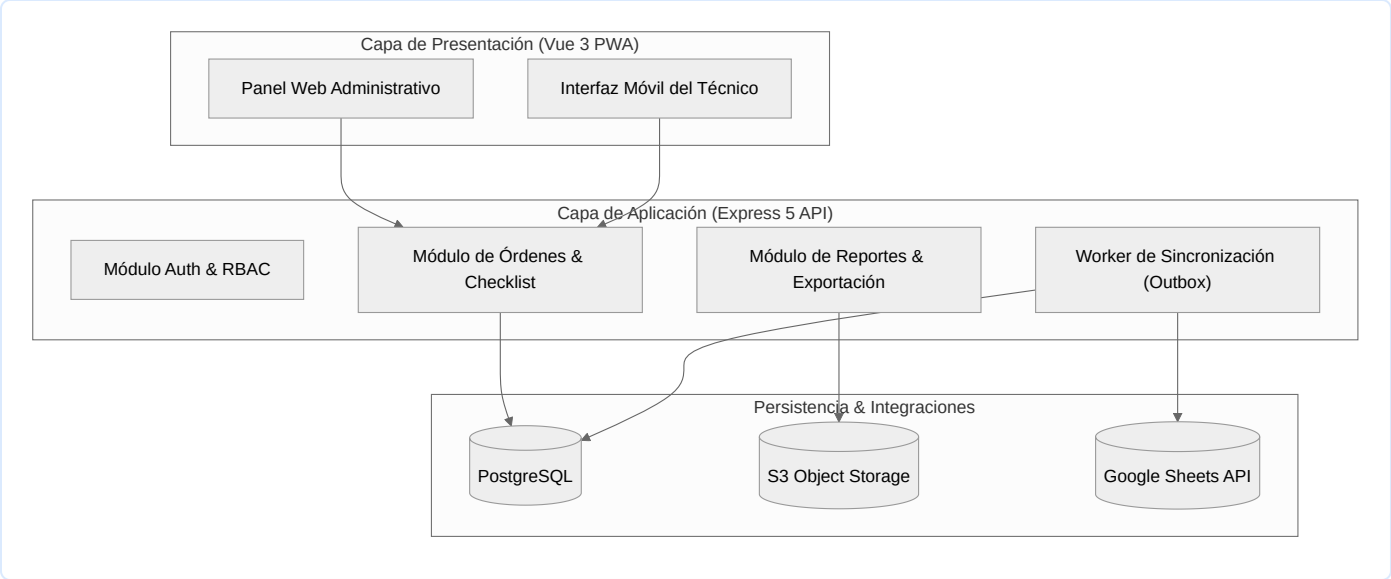
---

2. Descripción General

2.1 Perspectiva del Producto

El sistema opera bajo una arquitectura de monorepo con backend modular en Node.js (Express 5, TypeScript, Prisma ORM, PostgreSQL) y frontend reactivo en Vue 3 (Vite, Pinia, Naive UI, Tailwind CSS). Se integra de forma externa con:

- **Google Sheets API:** Autoridad de catálogos de Clientes y Equipos (`SALIDAS`).
- **Almacenamiento S3:** Repositorio privado de evidencias fotográficas y firmas digitales con URLs prefirmadas.



2.2 Perfiles de Usuario (Roles)

Rol	Descripción	Nivel de Acceso
Administrador (`ADMIN`)	Personal de operaciones y logística en oficina. Prepara órdenes, vincula clientes/unidades/equipos, asigna técnicos, gestiona catálogos locales, audita y consulta reportes globales.	Acceso total al sistema y módulos administrativos/reportes.
Técnico (`TECHNICIAN`)	Personal operativo en campo. Visualiza órdenes asignadas en interfaz móvil, valida hardware, llena checklist con fotos, levanta discrepancias, recoge firmas y finaliza servicios.	Acceso restringido únicamente a sus órdenes asignadas y flujo de ejecución.

2.3 Restricciones Generales

1. **Autoridad de Catálogos:** La creación y edición de clientes y equipos no se realiza en la aplicación; residen y se administran en Google Sheets.
2. **Columnas Protegidas en Sheets:** El sistema tiene prohibido modificar las columnas `Fecha de salida`, `Estado` y `Estado del servicio` en Google Sheets.
3. **Almacenamiento Seguro:** No se almacenan imágenes en base de datos; únicamente claves S3 de objetos privados. El acceso se realiza mediante URLs firmadas con expiración de 15 minutos.
4. **Modo Conectado:** El MVP asume conectividad a Internet para sincronización; no contempla soporte offline completo ni colas persistentes en IndexedDB.

---

3. Requisitos Específicos

3.1 Requisitos Funcionales (RF)

Módulo 1: Autenticación y Gestión de Usuarios

- **RF-01 (Autenticación Segura):** El sistema debe autenticar usuarios mediante correo electrónico y contraseña con hash Argon2id, retornando Access Token JWT (15 min) y Refresh Token (7 días) rotativo.
- **RF-02 (Control de Acceso Basado en Roles):** El sistema debe restringir endpoints y vistas según el rol (`ADMIN` o `TECHNICIAN`). Un técnico solo podrá consultar e interactuar con órdenes asignadas a su identificador.
- **RF-03 (Administración de Técnicos):** El administrador debe poder crear, editar y activar/desactivar perfiles de técnicos y asociarlos a cuentas de usuario del sistema.

Módulo 2: Integración con Google Sheets

- **RF-04 (Consulta de Catálogos en Sheets):** El sistema debe consultar el catálogo de clientes y de equipos disponibles (`SALIDAS`) mediante caché controlada con Redis/memoria y Google Sheets API.
- **RF-05 (Detección de Conflictos de Concurrencia):** Si la fila de un equipo o cliente cambia en Google Sheets durante la preparación de una orden, el sistema debe alertar al administrador sobre el conflicto de versión antes de confirmar.
- **RF-06 (Outbox y Reintentos de Sincronización):** Las actualizaciones dirigidas a Google Sheets deben registrarse en una tabla Outbox transaccional y procesarse asíncronamente con reintentos exponenciales y registro de errores.

Módulo 3: Gestión de Órdenes de Servicio

- **RF-07 (Creación de Órdenes Multilote):** El administrador debe poder crear una orden en estado `DRAFT`, asociarla a un cliente, registrar una o múltiples unidades vehiculares, y asignar uno o varios equipos por unidad.
- **RF-08 (Tipos de Servicio):** El sistema debe soportar tipos de servicio: `INSTALLATION`, `REVIEW`, `UNINSTALLATION` y `FAILED_APPOINTMENT`.
- **RF-09 (Asignación y Reasignación):** El administrador debe poder asignar y reasignar una orden a un técnico activo, transicionando su estado a `ASSIGNED`.
- **RF-10 (Ciclo de Vida de la Orden):** El sistema debe hacer cumplir la máquina de estados: `DRAFT` $\to$ `ASSIGNED` $\to$ `IN_PROGRESS` $\to$ `COMPLETED` / `REQUIRES_CORRECTION` / `FAILED_APPOINTMENT` / `CANCELLED`.

Módulo 4: Ejecución en Campo (Operación Técnica)

- **RF-11 (Vista Móvil del Técnico):** El técnico debe visualizar su listado de órdenes asignadas con detalles de cliente, ubicación, unidades y estatus.
- **RF-12 (Verificación de Equipamiento):** El técnico debe verificar individualmente cada equipo asignado a una unidad, marcándolo como `MATCHED` o `DISCREPANCY_REPORTED`.
- **RF-13 (Reporte de Discrepancias en Campo):** Si un equipo difiere físicamente (IMEI, serie, ID de equipo o unidad no coincidente), el técnico debe poder reportar los valores observados y continuar el servicio sin bloqueos.
- **RF-14 (Checklist Post-Servicio Obligatorio):** Para cada unidad, el técnico debe registrar el estado de 5 categorías obligatorias:
    1. Tablero (`DASHBOARD`)
    2. Cristales (`WINDOWS`)
    3. Accesorios (`ACCESSORIES`)

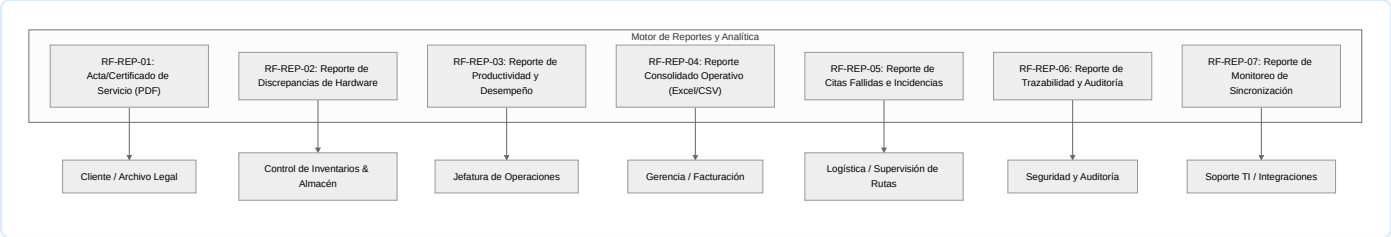
4. Molduras (`TRIMS`)
5. Carrocería (`BODYWORK`)

   - Estados permitidos: `GOOD` (Bien), `BAD` (Mal) o `NOT_APPLICABLE` (No aplica).
   - Si el estado es `BAD` , el campo de observación explicativa es **obligatorio**.

- **RF-15 (Evidencias Fotográficas Obligatorias):** Cada categoría del checklist requiere al menos una fotografía capturada y subida a S3 mediante URL prefirmada, independientemente de si se seleccionó `GOOD` , `BAD` o `NOT_APPLICABLE` .
- **RF-16 (Captura de Firma del Cliente):** La orden finalizada exige la captura digital de una única firma del cliente en pantalla, junto con el nombre completo y documento de la persona firmante.
- **RF-17 (Cierre de Cita Fallida):** Si la unidad no está disponible o el cliente no se presenta, el técnico puede registrar un cierre como `FAILED_APPOINTMENT` indicando obligatoriamente el motivo, omitiendo fotos, checklist y firma.

3.2 Requisitos Funcionales del Módulo de Reportes y Analítica (RF-REP)

Este módulo atiende la supervisión operativa, la auditoría técnica, la rendición de cuentas al cliente y el control administrativo.



RF-REP-01: Acta de Entrega y Certificado Técnico de Servicio (Formato PDF)

- **Descripción:** El sistema debe generar automáticamente un documento PDF formal y descargable al completarse la orden de servicio.
- **Entrada / Filtro:** Folio o ID único de la orden completada.
- **Contenido Requerido:**
  - Encabezado con logotipo corporativo de Bull Dog Solution, folio único, fecha y hora de inicio/cierre.
  - Datos del cliente, técnico responsable y taller/ubicación.
  - Detalle por unidad vehicular: identificación, placas/VIN, estado final.
  - Tabla de equipamiento verificado: IDs, números de serie, IMEIs y notas de discrepancia (si existieran).
  - Resultados del checklist vehicular clasificado en las 5 categorías con sus observaciones.
  - Mosaico de evidencias fotográficas incrustadas con sello de fecha/hora y categoría correspondiente.
  - Imagen de la firma digitalizada del cliente, nombre completo legible y leyenda de conformidad del servicio.
- **Formato de Salida:** PDF estándar A4/Carta descargable y consultable vía URL prefirmada de S3.

RF-REP-02: Reporte de Discrepancias de Equipamiento en Campo

- **Descripción:** Reporte especializado para auditoría de almacén e inventarios que consolida todos los desfases detectados entre lo programado en Google Sheets y lo encontrado físicamente en las unidades.
- **Criterios de Filtrado:** Rango de fechas, cliente, técnico, estado de la orden, tipo de discrepancia (IMEI, Serie, ID, Unidad).
- **Columnas Requeridas:**
  - Folio de la orden.
  - Fecha del reporte y nombre del técnico reportante.
  - Nombre del cliente y unidad vehicular afectada.
  - Datos esperados (según catálogo `SALIDAS`) vs. Datos reales capturados en campo.
  - Observaciones y estatus de atención administrativa.
- **Formato de Salida:** Vista tabular interactiva en pantalla con opción de exportación a CSV y Excel (XLSX).

RF-REP-03: Reporte de Productividad y Desempeño Operativo de Técnicos

- **Descripción:** Panel y reporte consolidado para evaluar el rendimiento del personal operativo.
- **Criterios de Filtrado:** Periodo de tiempo (diario, semanal, mensual), técnico individual o todos los técnicos.
- **Métricas e Indicadores:**
  - Total de órdenes asignadas vs. completadas.
  - Tasa de efectividad de visitas (Completadas vs. Citas Fallidas / Canceladas).
  - Tiempo promedio transcurrido entre asignación, inicio del servicio y cierre (`IN_PROGRESS` a `COMPLETED`).
  - Total de unidades y equipos instalados/revisados/desinstalados por técnico.
  - Conteo de discrepancias levantadas por técnico.
- **Formato de Salida:** Tablero con gráficos de barras/progreso y descarga de resumen en formato CSV/XLSX.

RF-REP-04: Reporte Consolidado de Órdenes y Servicios (Exportación General)

- **Descripción:** Reporte maestro para administración y finanzas que consolida todas las transacciones operativas del sistema.
- **Criterios de Filtrado:** Rango de fechas de creación/finalización, estado de la orden (`DRAFT`, `ASSIGNED`, `IN_PROGRESS`, `COMPLETED`, `FAILED_APPOINTMENT`, `CANCELLED`), cliente, tipo de servicio.
- **Columnas Requeridas:** Folio, Cliente, Tipo de Servicio, Técnico Asignado, Cantidad de Unidades, Cantidad de Equipos, Estado Actual, Fecha de Creación, Fecha de Cierre, Duración Total.
- **Formato de Salida:** Exportación asíncrona o sincrónica en formato Excel (XLSX) y CSV con soporte para UTF-8.

RF-REP-05: Reporte de Citas Fallidas y Cancelaciones

- **Descripción:** Informe operativo orientado a identificar causas recurrentes de no ejecución de servicios para cobro de visita o reasignación logística.
- **Criterios de Filtrado:** Rango de fechas, cliente, técnico responsable, motivo registrado.
- **Columnas Requeridas:**
  - Folio de la orden.
  - Cliente y ubicación de la cita.
  - Técnico asignado.

    - Motivo de la cita fallida (justificación obligatoria registrada por el técnico).
    - Fecha y hora exacta de registro.

- **Formato de Salida:** Vista en pantalla y exportación en CSV/Excel.

RF-REP-06: Reporte de Auditoría y Trazabilidad de Acciones

- **Descripción:** Registro inmutable de eventos del sistema para cumplimiento y control de cambios.
- **Criterios de Filtrado:** Rango de fechas, usuario responsable, entidad afectada (`Order`, `Unit`, `Equipment`, `User`), tipo de acción (`CREATE`, `UPDATE`, `STATUS_CHANGE`, `REASSIGN`).
- **Columnas Requeridas:** Timestamp ISO-8601, Usuario (ID y nombre), Dirección IP (si aplica), Acción realizada, Estado previo vs. Estado nuevo (Diff JSON).
- **Formato de Salida:** Visor de auditoría paginado con exportación de registros a CSV.

RF-REP-07: Reporte y Monitoreo de Sincronización con Google Sheets

- **Descripción:** Tablero y reporte técnico para soporte y administración que expone la salud del canal de sincronización bidireccional.
- **Métricas y Datos:**
    - Total de transacciones encoladas en Outbox (`PENDING`, `PROCESSING`, `SYNCED`, `FAILED`).
    - Detalle de fallas: mensaje de error de la API de Google, código HTTP, número de reintentos ejecutados.
    - Conflictos de versión detectados en catálogos de clientes o equipos.
    - Botón de acción para reintento manual asistido de tareas fallidas.
- **Formato de Salida:** Panel de diagnóstico técnico en tiempo real en la interfaz de administración.

3.3 Requisitos No Funcionales (RNF)

3.3.1 Rendimiento y Eficiencia

- **RNF-01 (Tiempos de Respuesta de la API):** El 95% de las peticiones REST estándar deben responder en menos de 300 ms bajo condiciones normales de red.
- **RNF-02 (Generación de Reportes PDF):** El tiempo de generación del PDF del acta de servicio no debe exceder los 3.5 segundos para órdenes con hasta 5 unidades y 25 imágenes.
- **RNF-03 (Optimización Móvil):** La aplicación móvil para técnicos debe tener un tamaño de bundle inicial menor a 1.5 MB y tiempos de renderizado menores a 1 segundo en dispositivos de gama media.

3.3.2 Seguridad y Protección de Datos

- **RNF-04 (Cifrado en Tránsito y Reposo):** Toda comunicación debe efectuarse sobre HTTPS/TLS 1.3. Las contraseñas deben cifrarse con Argon2id.
- **RNF-05 (Privacidad de Evidencias):** Los buckets de almacenamiento de imágenes y firmas deben ser estrictamente privados. La consulta de fotos se realizará exclusivamente mediante URLs firmadas con expiración máxima de 15 minutos.
- **RNF-06 (Prevención de Inyecciones y Validación):** Todas las cargas útiles entrantes (payloads) deben validarse de forma estricta en el servidor mediante esquemas Valibot.

3.3.3 Disponibilidad y Resiliencia

- **RNF-07 (Tolerancia a Fallos de Google Sheets):** La caída temporal o limitación de cuota de la API de Google Sheets no debe impedir la ejecución en campo de órdenes ya asignadas. Los eventos pendientes deben persistir en PostgreSQL mediante el patrón Outbox.
- **RNF-08 (Concurrencia Optimista):** El sistema debe prevenir sobreescrituras concurrentes en órdenes y sincronización mediante tokens de versión o timestamps de actualización.

3.3.4 Usabilidad y Accesibilidad

- **RNF-09 (Diseño Mobile-First):** La interfaz para técnicos debe ser 100% operable en pantallas táctiles de smartphones con diagonales desde 4.7 pulgadas, con botones y áreas táctiles de al menos 48x48 px.
- **RNF-10 (Claridad en Reporte de Checklist):** La captura de checklist y fotos debe presentar retroalimentación visual clara e inequívoca del progreso (p. ej., barra de avance de fotos capturadas).

---

4. Requisitos de Interfaz Externa

4.1 Interfaces de Usuario

- **Interfaz Web de Administración:** Desarrollada con Naive UI y Tailwind CSS, con diseño optimizado para escritorio (1280px+), tablas paginadas, filtros de búsqueda avanzada, modales de visualización de evidencias y tableros de reportes.
- **Interfaz Móvil del Técnico:** Aplicación web responsiva con comportamiento PWA, navegación guiada paso a paso, componente de lienzo táctil (`HTML5 Canvas`) para captura de firma y selector nativo de cámara para evidencias fotográficas.

4.2 Interfaces de Software y Terceros

- **Google Sheets API v4:**
  - Lectura de hoja de `Clientes` (Nombres, identificadores, contratos).
  - Lectura y actualización selectiva de hoja `SALIDAS` (catálogo de equipos, series e IMEIs).
- **Amazon S3 Compatible Storage:**
  - Generación de URLs prefirmadas (`PUT`) para carga directa desde cliente.
  - Generación de URLs prefirmadas (`GET`) para visualización y ensamblado de reportes PDF.
- **Motor de Renderizado PDF:**
  - Librería de servidor (p. ej., PDFKit o Puppeteer ligero) para la compilación de actas de servicio firmadas.

---

5. Matriz de Trazabilidad de Requisitos

ID Requisito	Descripción Sintética	Módulo Responsable	Componente de Sistema	Prioridad
RF-01	Autenticación JWT y sesiones	Seguridad	API Express / Auth Module	Alta
RF-02	RBAC (`ADMIN` vs `TECHNICIAN`)	Seguridad	API Express / Middleware	Alta
RF-04	Consulta a catálogos Sheets	Integración	API Express / Google Sheets Service	Alta
RF-07	Creación de órdenes multilote	Operaciones	API Express & Admin UI	Alta
RF-10	Máquina de estados de la orden	Núcleo	API Express / Orders Module	Crítica
RF-12	Verificación de equipos	Operación Campo	Technician Mobile UI	Alta
RF-13	Reporte de discrepancias	Operación Campo	Mobile UI & Orders Service	Media
RF-14	Checklist vehicular (5 categorías)	Operación Campo	Mobile UI & Checklist Service	Crítica
RF-15	Evidencias fotográficas (S3)	Operación Campo	S3 Adapter & Mobile UI	Crítica
RF-16	Firma digital del cliente	Operación Campo	Canvas Signature & S3 Adapter	Crítica
RF-17	Cierre como cita fallida	Operación Campo	Orders Module / Mobile UI	Alta
RF-REP-01	Acta técnica de servicio (PDF)	Reportes	Reports Engine & PDF Service	Crítica
RF-REP-02	Reporte de discrepancias	Reportes	Reports Engine & Admin UI	Media
RF-REP-03	Productividad de técnicos	Reportes	Reports Engine & Admin UI	Media
RF-REP-04	Consolidado general (XLSX/CSV)	Reportes	Reports Engine & Export Service	Alta
RF-REP-05	Reporte de citas fallidas	Reportes	Reports Engine & Admin UI	Media
RF-REP-06	Reporte de auditoría y trazabilidad	Reportes	Audit Service & Admin UI	Alta
RF-REP-07	Monitoreo de sincronización	Reportes	Outbox Worker & Admin UI	Alta
RNF-05	Privacidad de URLs en S3	Seguridad	Storage Service	Crítica
RNF-07	Resiliencia con Outbox	Arquitectura	Outbox Worker & PostgreSQL	Crítica

6. Apéndice: Estructura de Datos Base para Reportes

```
// Estructura de payload para generación del Reporte RF-REP-01 (Acta de Servicio PDF)
interface ServiceOrderReportData {
  folio: string;
  serviceType: 'INSTALLATION' | 'REVIEW' | 'UNINSTALLATION' | 'FAILED_APPOINTMENT';
  completedAt: string;
  client: {
    id: string;
    name: string;
  };
  technician: {
    id: string;
    name: string;
    workshop?: string;
  };
  units: Array<{
    id: string;
    identifier: string;
    plate?: string;
    vin?: string;
    checklist: Array<{
      category: 'DASHBOARD' | 'WINDOWS' | 'ACCESSORIES' | 'TRIMS' | 'BODYWORK';
      condition: 'GOOD' | 'BAD' | 'NOT_APPLICABLE';
      observation?: string;
      photos: Array<{
        url: string;
        uploadedAt: string;
      }>;
    }>;
    equipments: Array<{
      id: string;
      expectedImei: string;
      expectedSerial: string;
      verificationStatus: 'MATCHED' | 'DISCREPANCY_REPORTED';
      discrepancyNote?: string;
    }>;
  }>;
  signature: {
    signerName: string;
    signatureImageUrl: string;
    signedAt: string;
  };
}
```

Especificación técnica — Sistema de órdenes de servicio

Organización: Bull Dog Solution
Documento: Especificación técnica del MVP
Fecha: 2026-09-01
Versión: 0.2
Estado: Borrador técnico aprobado para iniciar implementación

1. Objetivo

Construir un sistema web responsivo que sustituya el flujo operativo actual en AppSheet para administrar órdenes de instalación, revisión y desinstalación de equipos en unidades vehiculares.

El sistema debe permitir que un administrador prepare completamente una orden, asigne un técnico, seleccione el cliente, registre las unidades y asigne uno o varios equipos a cada unidad. El técnico debe consultar sus órdenes, verificar la información, documentar el estado de cada unidad mediante checklist y fotografías, reportar discrepancias, obtener una firma final del cliente y cerrar el servicio.

Google Sheets continuará siendo la fuente oficial de los catálogos de clientes y equipos. La aplicación consultará esos datos mediante Google Sheets API y solo modificará las columnas expresamente autorizadas de la pestaña `SALIDAS`.

2. Alcance del MVP

2.1 Incluido

- Autenticación mediante JWT.
- Roles de administrador y técnico.
- Administración local de usuarios y técnicos.
- Consulta del catálogo de clientes en Google Sheets.
- Consulta y asignación de equipos desde Google Sheets.
- Creación, edición, asignación y seguimiento de órdenes.
- Varias unidades por orden.
- Varios equipos por unidad.
- Servicios de instalación, revisión, desinstalación y cita fallida.
- Checklist por unidad.
- Fotografías obligatorias por categoría del checklist.
- Reporte de discrepancias sin bloquear el trabajo del técnico.
- Una firma del cliente por orden.
- Reporte PDF de la orden de servicio completada, conforme a docs/service-order-report-spec.md.

- Almacenamiento privado compatible con S3.
- Auditoría de acciones relevantes.
- Registro y reintento de sincronizaciones con Google Sheets.
- Interfaz web responsiva y PWA básica.

2.2 Fuera del MVP

- Modo offline completo.
- Aplicaciones móviles nativas.
- Edición de clientes dentro de la aplicación.
- Alta o edición de equipos dentro de la aplicación.
- Facturación.
- Administración de cuentas bancarias.
- Cálculo o pago de tarifas a técnicos.
- Modificación automática de `Fecha de salida`, `Estado` o `Estado del servicio` en Google Sheets.
- Microservicios.

3. Reglas de negocio confirmadas

1. Una orden pertenece a un cliente y se asigna a un técnico.
2. Una orden contiene una o más unidades.
3. Una unidad puede tener varios equipos.
4. El administrador registra previamente la orden, las unidades y los equipos.
5. El técnico no crea unidades ni equipos desde cero.
6. El checklist se realiza después del servicio.
7. Las categorías iniciales del checklist son:
    - Tablero.
    - Cristales.
    - Accesorios.
    - Molduras.
    - Carrocería.
8. Cada categoría admite `Bien`, `Mal` o `No aplica`.
9. Cada categoría requiere al menos una fotografía, incluso cuando se marque `No aplica`, hasta que el negocio decida lo contrario.
10. Cuando una categoría se marque `Mal`, la observación es obligatoria.
11. Para finalizar un servicio normal, todas las unidades deben tener:
    - Equipos verificados.
    - Checklist completo.
    - Al menos una fotografía por categoría.
12. Se requiere una sola firma del cliente por toda la orden.

13. La firma requiere el nombre de la persona que firma.
14. Si el IMEI, ID, serie o unidad no coincide, el técnico reporta una discrepancia y continúa.
15. Una cita fallida exige solamente un motivo y no requiere checklist, fotografías ni firma.
16. Google Sheets conserva la autoridad sobre los catálogos de clientes y equipos.
17. La aplicación no modifica las columnas `Fecha de salida`, `Estado` ni `Estado del servicio`.

4. Stack tecnológico

Capa	Tecnología
Lenguaje	TypeScript
Frontend	Vue 3 + Vite
Enrutamiento	Vue Router
Estado de cliente	Pinia
Estado de servidor y caché	TanStack Query para Vue
Cliente HTTP	Axios
Validación	Valibot
Componentes UI	Naive UI
Estilos	Tailwind CSS mediante plugin de Vite
Backend	Node.js + Express 5
ORM	Prisma
Base de datos	PostgreSQL
Autenticación	JWT con access y refresh tokens
Hash de contraseñas	Argon2id
Objetos	API compatible con Amazon S3
API de Google	`googleapis` / Google Sheets API v4
Seguridad HTTP	Helmet, CORS y rate limiting
Logs HTTP	Morgan
Logs de aplicación	Winston con salida estructurada JSON
Documentación API	OpenAPI 3.1 + Swagger UI
Resiliencia externa	Timeouts, reintentos controlados y circuit breaker
Utilidades Vue	VueUse cuando evite implementaciones propias

Capa	Tecnología
Pruebas	Vitest, Supertest y Playwright
Contenedores	Docker
Gestor de paquetes	pnpm

Las versiones exactas se fijarán en el lockfile al crear el proyecto. Se usarán versiones estables compatibles con la versión LTS de Node.js seleccionada al iniciar la implementación.

4.1 Paquetes principales previstos

```
# Compartidos
typescript valibot @valibot/to-json-schema axios

# API
express prisma @prisma/client jose argon2
cors helmet cookie-parser express-rate-limit morgan winston
swagger-ui-express opossum googleapis
@aws-sdk/client-s3 @aws-sdk/s3-request-presigner

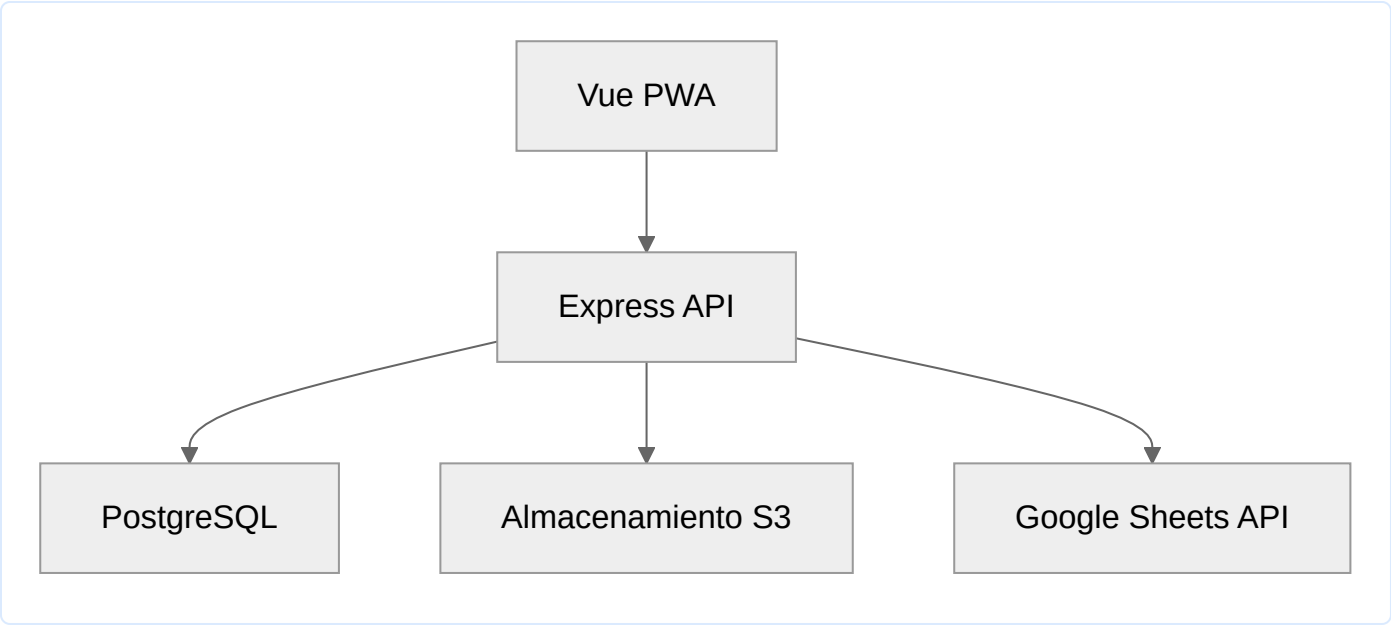
# Web
vue vue-router pinia @tanstack/vue-query
naive-ui tailwindcss @tailwindcss/vite @vueuse/core

# Pruebas
vitest supertest @playwright/test
```

La lista es una intención arquitectónica; antes de instalar se eliminarán paquetes redundantes y se comprobará compatibilidad entre versiones.

5. Arquitectura

Se implementará un monolito modular. El frontend y el backend vivirán en un monorepo, pero se construirán como aplicaciones separadas.



5.1 Responsabilidades

Vue: interfaz, navegación, captura de cámara, firma, validación inmediata y consumo de la API.

Express: autenticación, autorización, validación final, reglas de negocio, Prisma, Google Sheets, URLs firmadas para objetos, auditoría y manejo de errores.

PostgreSQL: datos transaccionales, órdenes, snapshots externos, checklist, referencias de evidencias, sesiones, auditoría y estado de sincronización.

S3: bytes de fotografías y firmas. El bucket será privado.

Google Sheets: catálogo oficial de clientes y equipos y datos actuales de asignación permitidos.

5.2 Principios de diseño

- **Monolito modular:** un solo despliegue de API, dividido por dominios de negocio.
- **Slicing vertical:** cada módulo contiene sus rutas, casos de uso, persistencia, contratos y pruebas.
- **Dependencias hacia adentro:** controladores y adaptadores dependen de servicios/casos de uso, no al revés.
- **Controladores delgados:** traducen HTTP a una llamada de aplicación; no contienen reglas de negocio.
- **Integraciones detrás de puertos:** Google Sheets y S3 se consumen mediante interfaces reemplazables y adaptadores concretos.
- **Contratos compartidos:** Valibot define los DTOs; Prisma define persistencia y no se expone al frontend.
- **Estado remoto separado del estado de UI:** TanStack Query administra datos del servidor; Pinia solo estado global de interfaz y sesión.
- **Abstracción progresiva:** una vista o componente se mueve a `shared` después de comprobar reutilización real, no por anticipación.
- **Sin tiempo real innecesario:** el MVP no incluye Socket.IO. Se añadirá únicamente si aparece un caso que no pueda resolverse con invalidación de consultas o polling moderado.

6. Estructura del repositorio

```
service-orders/
├── apps/
│   ├── web/
│   │   ├── src/
│   │   │   ├── app/
│   │   │   ├── modules/
│   │   │   │   ├── auth/
│   │   │   │   ├── orders/
│   │   │   │   ├── technicians/
│   │   │   │   ├── external-catalogs/
│   │   │   │   └── service-execution/
│   │   │   ├── shared/
│   │   │   │   ├── components/
│   │   │   │   ├── composables/
│   │   │   │   ├── http/
│   │   │   │   ├── layouts/
│   │   │   │   └── utils/
│   │   │   ├── router/
│   │   │   ├── stores/
│   │   │   └── main.ts
│   │   └── vite.config.ts
│   └── api/
│       ├── src/
│       │   ├── config/
│       │   ├── docs/
│       │   ├── events/
│       │   ├── middleware/
│       │   ├── modules/
│       │   ├── resilience/
│       │   ├── shared/
│       │   ├── app.ts
│       │   └── server.ts
│       └── tsconfig.json
├── packages/
│   ├── contracts/
│   ├── eslint-config/
│   └── typescript-config/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── infra/
│   └── docker/
├── docker-compose.yml
├── pnpm-workspace.yaml
└── package.json
```

7. Módulos del backend

```
modules/
├── auth
├── users
├── technicians
├── external-clients
├── external-equipment
├── orders
├── units
├── order-equipment
├── checklists
├── evidence
├── signatures
├── discrepancies
├── google-sheets
├── object-storage
├── audit
└── health
```

Cada módulo podrá contener `router`, `controller`, `service`, `repository`, esquemas Valibot, tipos y pruebas. Los controladores serán delgados; las reglas vivirán en servicios.

7.1 Estructura estándar de un módulo backend

```
modules/{domain}/
├── {domain}.router.ts
├── {domain}.controller.ts
├── {domain}.service.ts
├── {domain}.repository.ts
├── {domain}.policy.ts
├── {domain}.schemas.ts
├── {domain}.types.ts
├── {domain}.errors.ts
├── {domain}.openapi.ts
└── __tests__/
```

- `router`: rutas y composición de middleware.
- `controller`: adaptación HTTP, sin decisiones de negocio.
- `service`: casos de uso y transacciones.
- `repository`: consultas Prisma específicas del dominio.
- `policy`: autorización contextual, por ejemplo pertenencia de una orden al técnico.
- `schemas`: contratos Valibot del módulo.
- `openapi`: operación, ejemplos y respuestas para la documentación.
- `errors`: errores de dominio traducidos por el manejador global.

No se crearán repositorios genéricos universales. Prisma se encapsulará solo cuando la abstracción proteja reglas, facilite pruebas o evite consultas duplicadas.

8. Modelo de datos

8.1 Enumeraciones

```
UserRole:
  ADMIN
  TECHNICIAN

OrderServiceType:
  INSTALLATION
  REVIEW
  UNINSTALLATION
  FAILED_APPOINTMENT

OrderStatus:
  DRAFT
  ASSIGNED
  IN_PROGRESS
  REQUIRES_CORRECTION
  COMPLETED
  FAILED_APPOINTMENT
  CANCELLED

UnitStatus:
  PENDING
  IN_PROGRESS
  COMPLETED

ChecklistCategory:
  DASHBOARD
  WINDOWS
  ACCESSORIES
  TRIMS
  BODYWORK

ChecklistCondition:
  GOOD
  BAD
  NOT_APPLICABLE

EquipmentVerificationStatus:
  PENDING
  MATCHED
  DISCREPANCY_REPORTED

UploadStatus:
  PENDING
  COMPLETED
  REJECTED
  DELETED

SyncStatus:
  PENDING
  PROCESSING
  SYNCED
  FAILED
```

8.2 Entidades principales

User

- `id: UUID`
- `email: string`, único y normalizado.
- `passwordHash: string`.
- `name: string`.
- `role: UserRole`.
- `technicianId: UUID?`.
- `active: boolean`.
- `createdAt`, `updatedAt`.

Technician

- `id: UUID`.
- `name: string`.
- `workshop: string?`.
- `address: string?`.
- `city: string?`.
- `state: string?`.
- `phone: string?`.
- `active: boolean`.
- `createdAt`, `updatedAt`.

Los datos fiscales, bancarios y tarifas quedan fuera del MVP.

Order

- `id: UUID`.
- `folio: string`, único.
- `serviceType: OrderServiceType`.
- `status: OrderStatus`.
- `clientSheetId: string`.
- `clientNameSnapshot: string`.
- `technicianId: UUID`.
- `scheduledAt: timestamptz`.
- `isHomeService: boolean`.
- `serviceAddress: string?`.
- `generalNotes: string?`.
- `failedReason: string?`.
- `version: integer`, para concurrencia optimista.
- `createdById: UUID`.
- `startedAt`, `completedAt`, `cancelledAt`.
- `createdAt`, `updatedAt`.

El número de semana se deriva de `scheduledAt` y no se almacena como dato editable.

OrderUnit

- `id: UUID`.
- `orderId: UUID`.
- `label: string`.
- `makeModel: string`.
- `year: integer?`.
- `serialNumber: string`.
- `status: UnitStatus`.
- `createdAt`, `updatedAt`.

El número de serie debe ser único dentro de una orden. La unicidad global se evaluará con los datos reales antes de imponerla.

OrderEquipment

- `id: UUID`.
- `orderUnitId: UUID`.
- `sheetName: string`, inicialmente `SALIDAS`.
- `sheetRowHint: integer?`, solo como optimización, nunca como identidad.
- `externalId: string?`, columna D.
- `imei: string?`, columna E.
- `externalKey: string`, identidad interna derivada.
- `brandSnapshot: string?`.
- `modelSnapshot: string?`.
- `lineSnapshot: string?`.
- `iccidSnapshot: string?`.
- `carrierSnapshot: string?`.
- `equipmentConditionSnapshot: string?`.
- `serviceStatusSnapshot: string?`.
- `rawSnapshot: jsonb`.
- `verificationStatus: EquipmentVerificationStatus`.
- `verifiedAt: timestamptz?`.
- `verifiedById: UUID?`.
- `createdAt`, `updatedAt`.

Un equipo no puede estar asignado a dos órdenes activas dentro de la aplicación. Se agregará un índice único parcial mediante una migración SQL cuando la restricción no pueda expresarse completamente en Prisma.

Checklist

- `id: UUID`.
- `orderUnitId: UUID`, único.
- `completedById: UUID?`.

- `completedAt: timestamptz?`.
- `createdAt`, `updatedAt`.

ChecklistItem

- `id: UUID`.
- `checklistId: UUID`.
- `category: ChecklistCategory`.
- `condition: ChecklistCondition?`.
- `notes: string?`.
- `createdAt`, `updatedAt`.

Restricción única: `checklistId + category`.

Evidence

- `id: UUID`.
- `checklistItemId: UUID`.
- `objectKey: string`, único.
- `mimeType: string`.
- `sizeBytes: integer`.
- `checksum: string?`.
- `width: integer?`.
- `height: integer?`.
- `status: UploadStatus`.
- `uploadedById: UUID`.
- `createdAt`, `completedAt`, `deletedAt`.

Signature

- `id: UUID`.
- `orderId: UUID`, único.
- `signerName: string`.
- `objectKey: string`, único.
- `mimeType: string`.
- `sizeBytes: integer`.
- `signedAt: timestamptz`.
- `capturedById: UUID`.
- `createdAt`.

Discrepancy

- `id: UUID`.
- `orderId: UUID`.
- `orderUnitId: UUID`.
- `orderEquipmentId: UUID?`.

- `type: string`.
- `description: string`.
- `expectedSnapshot: jsonb`.
- `observedSnapshot: jsonb`.
- `reportedById: UUID`.
- `reportedAt: timestamptz`.
- `resolvedAt: timestamptz?`.
- `resolvedById: UUID?`.
- `resolutionNotes: string?`.

La discrepancia no impide que el técnico continúe ni que cierre la orden si el resto de los requisitos se cumple.

RefreshSession

- `id: UUID`.
- `userId: UUID`.
- `jti: UUID`, único.
- `tokenHash: string`.
- `userAgent: string?`.
- `ipAddress: inet?`.
- `expiresAt: timestamptz`.
- `revokedAt: timestamptz?`.
- `replacedByJti: UUID?`.
- `createdAt`.

AuditLog

- `id: UUID`.
- `actorUserId: UUID?`.
- `action: string`.
- `entityType: string`.
- `entityId: string?`.
- `before: jsonb?`.
- `after: jsonb?`.
- `requestId: string?`.
- `ipAddress: inet?`.
- `createdAt`.

No se registrarán contraseñas, JWT, credenciales, contenido binario ni URLs firmadas.

SheetSyncOperation

- `id: UUID`.
- `orderId: UUID?`.
- `operationType: string`.
- `idempotencyKey: string`, único.

- `payload: jsonb`.
- `status: SyncStatus`.
- `attempts: integer`.
- `lastErrorCode: string?`.
- `lastErrorMessage: string?` sanitizado.
- `nextAttemptAt: timestamptz?`.
- `createdAt`, `updatedAt`, `syncedAt`.

9. Estados y transiciones

9.1 Orden normal

```
DRAFT → ASSIGNED → IN_PROGRESS → COMPLETED
                 → REQUIRES_CORRECTION → IN_PROGRESS
       → CANCELLED
```

9.2 Cita fallida

```
ASSIGNED → IN_PROGRESS → FAILED_APPOINTMENT
```

El cierre como cita fallida requiere únicamente `failedReason` no vacío.

9.3 Reglas de transición

- Solo un administrador crea, edita, asigna, reasigna o cancela órdenes.
- Un técnico solo inicia y trabaja órdenes asignadas a él.
- Una orden completada es inmutable para el técnico.
- Un administrador puede marcarla `REQUIRES_CORRECTION` conservando auditoría.
- Reabrir una orden no elimina evidencias ni firmas anteriores; cualquier reemplazo queda auditado.
- Las transiciones se validan en el servicio del backend y nunca dependen solo del frontend.

10. Integración con Google Sheets

10.1 Autenticación

- La integración se ejecuta exclusivamente en el backend.
- Se utilizará una cuenta de servicio de Google Cloud.
- El documento se compartirá con el correo de la cuenta de servicio.
- Las credenciales se guardarán como secretos del entorno.
- El frontend nunca recibirá credenciales ni tokens de Google.
- Se otorgará el alcance mínimo necesario para leer y actualizar el documento configurado.

10.2 Pestaña `Clientes`

Columna	Encabezado	Uso
A	`IDCliente`	Identificador externo estable
B	`Cliente`	Nombre visible

La pestaña será de solo lectura. La orden guardará `IDCliente` y un snapshot del nombre.

10.3 Pestaña `SALIDAS`

Columna	Encabezado	Permiso
A	Fecha de salida	Solo lectura
B	Marca	Solo lectura
C	Modelo	Solo lectura
D	ID	Solo lectura
E	IMEI	Solo lectura
F	Línea	Solo lectura
G	ICCID	Solo lectura
H	Compañía	Solo lectura
I	Estado	Solo lectura; valores `NUEVO` o `USADO`
J	Cliente	Escritura al asignar
K	Estado del Servicio	Solo lectura; por ejemplo `INSTALADO` o `STOCK TECNICO`
L	UNIDAD	Escritura al asignar
M	MARCA/MODELO DE LA UNIDAD	Escritura al asignar
N	AÑO	Escritura al asignar
O	NUMERO DE SERIE	Escritura al asignar
P	Tipo de servicio	Escritura solo si forma parte explícita de la asignación

La aplicación nunca escribirá rangos completos de una fila. Solo actualizará `J`, `L`, `M`, `N`, `O` y, cuando corresponda, `P`.

El valor de P no se derivará automáticamente del enum `OrderServiceType` hasta definir su catálogo real. En la hoja se observaron valores como `ESTANDAR` y `OBD/SCAN`, que representan un concepto diferente a instalación, revisión, desinstalación o cita fallida.

10.4 Identificación de equipos

La fila no es una identidad estable porque puede cambiar al ordenar o insertar registros.

Regla de identidad:

1. Usar `ID` cuando tenga un valor válido y no sea `PENDIENTE` o `N/A`.
2. Validar además el IMEI cuando exista.
3. Si el ID no es válido, usar el IMEI como llave temporal.
4. Si no hay ID ni IMEI único, mostrar `Datos incompletos` y no permitir asignar.
5. Guardar `sheetRowHint` solo para acelerar la búsqueda.
6. Antes de escribir, volver a localizar la fila y comprobar ID, IMEI y snapshot esperado.

Se recomienda agregar en el futuro una columna protegida `SISTEMA_ID` con UUID inmutable, pero no es requisito del MVP.

10.5 Lectura y caché

- Los clientes y equipos se consultan desde Google Sheets.
- El backend podrá usar caché de corta duración para búsquedas.
- La caché no será autoritativa.
- Antes de asignar se realizará una lectura fresca.
- La UI mostrará cuándo se obtuvo el catálogo y permitirá recargarlo.

10.6 Escritura y concurrencia

Antes de modificar una fila:

1. Localizar nuevamente el registro por ID/IMEI.
2. Comparar las columnas relevantes con el snapshot mostrado al administrador.
3. Comprobar que el equipo no esté en otra orden activa de PostgreSQL.
4. Si cambió externamente, responder `409 SHEET_ROW_CONFLICT`.
5. Actualizar únicamente las celdas autorizadas mediante `spreadsheets.values.batchUpdate`.
6. Guardar la respuesta y el nuevo snapshot.

PostgreSQL y Google Sheets no comparten una transacción distribuida. Se utilizará `SheetSyncOperation` con idempotencia, estados y reintentos. Una orden no se mostrará como completamente sincronizada mientras exista una operación fallida.

10.7 Reintentos

- Reintentos con backoff exponencial y jitter.
- Máximo configurable de intentos automáticos.
- Los errores de autenticación o permisos no se reintentan indefinidamente.
- El administrador puede reintentar una operación fallida.
- Cada operación incluye una `idempotencyKey`.
- Los errores se sanitizan antes de guardarse o mostrarse.

11. API REST

Prefijo: `/api/v1`.

11.1 Convenciones

- JSON en `camelCase`.
- Fechas en ISO 8601 UTC.
- IDs internos UUID.
- Paginación mediante `page` y `pageSize`; los listados de crecimiento alto podrán migrar a cursor sin cambiar el resto del contrato.
- Filtros comunes: `search`, `status`, `from`, `to`, `sortBy` y `sortOrder`.
- Cada endpoint declara una allowlist de filtros y campos ordenables; nunca se traduce una query arbitraria a Prisma.
- Un middleware compartido normaliza paginación, límites y ordenamiento.
- `pageSize` tendrá un máximo configurable, inicialmente 100.
- `requestId` en cada respuesta.
- Valibot valida params, query y body.
- Los errores no exponen stack traces en producción.

Respuesta exitosa:

```
{
  "data": {},
  "meta": {
    "requestId": "uuid",
    "pagination": {
      "page": 1,
      "pageSize": 25,
      "total": 120,
      "totalPages": 5
    }
  }
}
```

Respuesta de error:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "La solicitud contiene datos inválidos",
    "fields": {
      "email": ["Correo inválido"]
    },
    "requestId": "uuid"
  }
}
```

11.2 Autenticación

Método	Ruta	Descripción
POST	`/auth/login`	Iniciar sesión
POST	`/auth/refresh`	Rotar refresh token y emitir access token
POST	`/auth/logout`	Revocar sesión actual
POST	`/auth/logout-all`	Revocar todas las sesiones
GET	`/auth/me`	Usuario autenticado

11.3 Usuarios y técnicos

Método	Ruta	Rol
GET	`/users`	Admin
POST	`/users`	Admin
PATCH	`/users/:id`	Admin
GET	`/technicians`	Admin
POST	`/technicians`	Admin
PATCH	`/technicians/:id`	Admin

11.4 Catálogos externos

Método	Ruta	Descripción
GET	`/external/clients?search=`	Buscar clientes en `Clientes`
GET	`/external/equipment?search=`	Buscar equipos en `SALIDAS`
GET	`/external/equipment/:externalKey`	Obtener y revalidar equipo
POST	`/external/refresh`	Invalidar caché; admin

11.5 Órdenes

Método	Ruta	Descripción
GET	`/orders`	Listar según rol
POST	`/orders`	Crear borrador; admin
GET	`/orders/:id`	Obtener detalle

Método	Ruta	Descripción
PATCH	`/orders/:id`	Editar borrador; admin
POST	`/orders/:id/assign`	Asignar y sincronizar; admin
POST	`/orders/:id/start`	Iniciar; técnico asignado
POST	`/orders/:id/cancel`	Cancelar; admin
POST	`/orders/:id/require-correction`	Solicitar corrección; admin
POST	`/orders/:id/complete`	Finalizar servicio normal
POST	`/orders/:id/fail-appointment`	Cerrar como cita fallida
GET	`/orders/:id/audit`	Historial; admin

11.6 Unidades y equipos

Método	Ruta	Descripción
POST	`/orders/:id/units`	Agregar unidad; admin
PATCH	`/orders/:id/units/:unitId`	Editar unidad; admin
DELETE	`/orders/:id/units/:unitId`	Eliminar en borrador; admin
POST	`/orders/:id/units/:unitId/equipment`	Asignar equipos; admin
DELETE	`/orders/:id/units/:unitId/equipment/:equipmentId`	Desasignar en borrador; admin
POST	`/orders/:id/units/:unitId/equipment/:equipmentId/verify`	Confirmar coincidencia
POST	`/orders/:id/units/:unitId/equipment/:equipmentId/discrepancy`	Reportar diferencia

11.7 Checklist y evidencias

Método	Ruta	Descripción
PUT	`/orders/:id/units/:unitId/checklist`	Guardar resultados
POST	`/orders/:id/units/:unitId/checklist/:category/evidence/presign`	Crear subida autorizada
POST	`/evidence/:id/complete`	Confirmar subida
DELETE	`/evidence/:id`	Eliminación lógica autorizada

11.8 Firma

Método	Ruta	Descripción
POST	`/orders/:id/signature/presign`	Preparar subida de firma
POST	`/orders/:id/signature/complete`	Confirmar firma y nombre
GET	`/orders/:id/signature/url`	URL temporal autorizada

11.9 Sincronización

Método	Ruta	Descripción
GET	`/orders/:id/sync`	Estado de sincronización
POST	`/orders/:id/sync/retry`	Reintentar; admin

11.10 OpenAPI y Swagger UI

- La API publicará OpenAPI 3.1 en `/api/v1/openapi.json`.
- Swagger UI estará disponible en `/api/docs` y podrá deshabilitarse en producción.
- Los esquemas JSON reutilizables se generarán desde Valibot mediante `@valibot/to-json-schema` cuando la conversión sea compatible.
- Operaciones, códigos HTTP, seguridad, ejemplos y errores se declararán junto al módulo correspondiente.
- Los casos que no puedan convertirse automáticamente se documentarán de forma explícita y se cubrirán con pruebas de contrato.
- CI validará el documento OpenAPI y detectará referencias rotas.
- El contrato documentado nunca reemplaza la validación en runtime.

12. Valibot y contratos compartidos

`packages/contracts` contendrá DTOs y esquemas reutilizables:

```
LoginSchema
CreateOrderSchema
UpdateOrderSchema
AssignOrderSchema
CreateUnitSchema
AssignEquipmentSchema
VerifyEquipmentSchema
ReportDiscrepancySchema
SaveChecklistSchema
PresignEvidenceSchema
CompleteEvidenceSchema
CompleteSignatureSchema
CompleteOrderSchema
FailAppointmentSchema
```

Reglas:

- Todo dato externo se considera `unknown` hasta pasar por Valibot.
- El backend vuelve a validar aunque Vue ya lo haya hecho.
- Los tipos se infieren desde los esquemas.
- Los esquemas incluyen descripciones y metadatos útiles para formularios y OpenAPI.
- `@valibot/to-json-schema` genera los componentes JSON Schema compatibles para la especificación OpenAPI.
- Los modelos Prisma no se exportan como contratos del frontend.
- Las variables de entorno también se validan con Valibot al iniciar.
- La aplicación falla al arrancar si falta una variable obligatoria.

13. JWT y sesiones

13.1 Tokens

Access token:

- JWT firmado.
- Duración sugerida: 15 minutos.
- Se conserva en memoria en Vue.
- Se envía como `Authorization: Bearer <token>`.
- Claims mínimos: `sub`, `role`, `iat`, `exp`, `iss`, `aud`.

Refresh token:

- JWT firmado con clave distinta o propósito criptográfico distinto.
- Duración sugerida: 30 días.
- Cookie `HttpOnly`, `Secure` y `SameSite` según el despliegue.
- Incluye `jti`.
- Su hash se registra en `RefreshSession`.
- Se rota en cada uso.

13.2 Axios

Se creará una instancia única de Axios que:

- Define `baseURL`.
- Usa `withCredentials: true`.
- Agrega el access token.
- Al recibir `401`, ejecuta un único refresh compartido.
- Reintenta una sola vez la solicitud original.
- Evita ciclos infinitos.
- Limpia el estado cuando la renovación falla.
- Normaliza los errores de la API.

- Permite cancelar peticiones con `AbortController`.

No se guardarán JWT en `localStorage` ni `sessionStorage`.

13.3 Contraseñas

- Hash Argon2id.
- Nunca se registran en logs.
- Política inicial configurable de longitud mínima.
- El login responde de forma genérica para usuario inexistente o contraseña incorrecta.
- Rate limiting por IP y por cuenta.

14. Autorización

14.1 Administrador

- Administrar usuarios y técnicos.
- Consultar catálogos externos.
- Crear y editar órdenes.
- Registrar unidades.
- Asignar equipos y técnicos.
- Consultar todas las órdenes.
- Cancelar, reabrir o solicitar corrección.
- Consultar auditoría y sincronización.

14.2 Técnico

- Consultar únicamente órdenes asignadas.
- Iniciar una orden.
- Ver cliente, unidades y equipos asignados.
- Verificar equipos.
- Reportar discrepancias.
- Completar checklist.
- Adjuntar y eliminar sus evidencias mientras la orden esté abierta.
- Capturar nombre y firma del cliente.
- Finalizar o marcar cita fallida.

La autorización se aplica en backend y las consultas Prisma incluyen el filtro de pertenencia correspondiente.

14.3 Modelo de permisos

El MVP conservará dos roles (`ADMIN` y `TECHNICIAN`) y definirá permisos como capacidades de código, por ejemplo `orders:create`, `orders:read:any`, `orders:read:assigned`, `orders:complete:assigned` y `sync:retry`.

- Las rutas declaran la capacidad requerida.

- Las políticas del dominio validan además contexto y propiedad del recurso.
- No se crearán inicialmente tablas genéricas `Module`, `Operation`, `Action` y `Permission`.
- Si aparecen más perfiles configurables, las capacidades podrán migrarse a RBAC persistido sin cambiar los controladores.
- JWT incluye el rol, no una lista larga de permisos potencialmente obsoleta.

15. Evidencias y almacenamiento S3

15.1 Flujo de subida

1. Vue comprime la imagen y solicita una URL firmada.
2. Express verifica usuario, orden, unidad y categoría.
3. Express crea `Evidence` con estado `PENDING`.
4. Express genera una URL prefirmada de corta duración.
5. Vue sube directamente al almacenamiento.
6. Vue llama a `/evidence/:id/complete`.
7. Express consulta metadatos del objeto y valida tamaño y tipo.
8. Express cambia el estado a `COMPLETED`.

15.2 Reglas

- Bucket privado.
- No almacenar blobs o Base64 en PostgreSQL.
- Tipos iniciales: JPEG, PNG y WebP.
- Tamaño máximo configurable; sugerido 10 MiB por evidencia.
- La firma se almacena como PNG, con máximo sugerido de 1 MiB.
- URLs firmadas de subida con expiración corta, sugerida de 5 minutos.
- Nombres generados por el servidor; nunca confiar en el nombre del usuario.
- Verificar `Content-Type`, longitud y existencia antes de completar.
- Aplicar política de limpieza para objetos huérfanos.
- Descargas mediante URLs temporales o proxy autorizado.

15.3 Estructura de claves

```
orders/{orderId}/units/{unitId}/checklist/{category}/{evidenceId}.{ext}
orders/{orderId}/signature/{signatureId}.png
```

16. Frontend Vue

El frontend seguirá una arquitectura modular por dominios. Cada módulo será autónomo y solo publicará su API pública desde un archivo `index.ts`; otros módulos no importarán archivos internos por rutas profundas.

16.1 Estructura estándar de un módulo

```
modules/{domain}/
├── api/
│   └── {domain}.api.ts
├── components/
├── composables/
├── routes/
│   └── index.ts
├── schemas/
├── views/
├── types.ts
└── index.ts
```

- `api`: funciones Axios específicas del dominio.
- `components`: UI exclusiva del módulo.
- `composables`: orquestación de formularios, consultas y acciones.
- `routes`: rutas cargadas de forma diferida.
- `schemas`: reexporta o extiende contratos Valibot compartidos.
- `views`: pantallas asociadas a rutas.
- `index.ts`: superficie pública del módulo.

No se mantendrá una segunda copia completa de datos remotos en Pinia. Las funciones de `api` alimentarán a TanStack Query y las mutaciones invalidarán las claves de consulta correspondientes.

16.2 Vistas de administrador

- Inicio de sesión.
- Dashboard.
- Lista y filtros de órdenes.
- Creación de orden en pasos.
- Selección de cliente desde Google Sheets.
- Registro de unidades.
- Selección de varios equipos por unidad.
- Asignación de técnico.
- Detalle, evidencias, firma y auditoría de orden.
- Directorio de técnicos.
- Estado y reintento de sincronización.

16.3 Vistas del técnico

- Mis órdenes.
- Detalle de orden.
- Inicio del servicio.
- Lista y progreso de unidades.
- Equipos esperados y verificación.
- Formulario de discrepancia.

- Checklist móvil por unidad.
- Cámara y galería de evidencias.
- Captura de nombre y firma.
- Resumen y cierre.
- Cita fallida.

16.4 Estado

- Pinia: usuario, access token y estado estrictamente global de UI.
- TanStack Query: órdenes, catálogos y datos remotos.
- Estado temporal de formularios: componentes/composables.
- No duplicar respuestas completas de API en Pinia.
- Los formularios editan un borrador local y conservan un snapshot para cancelar cambios sin volver a consultar.
- Los filtros pertenecen a la URL cuando deban sobrevivir recargas, navegación o enlaces compartidos.
- Los loaders globales se reservan para transiciones amplias; cada consulta muestra su propio estado de carga.
- Errores y confirmaciones se normalizan mediante un servicio de notificaciones compartido.

16.5 Sistema visual y componentes reutilizables

- Naive UI proveerá tablas, formularios, modales, paginación, notificaciones y controles accesibles.
- Tailwind CSS se utilizará para layout, espaciado y ajustes responsivos, evitando sobrescribir internamente componentes de Naive UI sin necesidad.
- El tema de Naive UI centralizará colores, tipografía y estados de la marca.
- Se crearán wrappers delgados únicamente para patrones repetidos, por ejemplo `BaseListPage`, `BaseDetailPage`, `PageHeader`, `AsyncState` y `ConfirmAction`.
- Los wrappers usarán props y slots; no conocerán dominios como órdenes o técnicos.
- Un componente pasará a `shared` después de tener al menos dos consumidores reales o representar una regla visual global.
- Se preferirá `<script setup lang="ts">` y Composition API.

16.6 Flujo de datos del frontend

```
View → Composable → Query/Mutation → API module → Axios → Express
     ← estado reactivo y errores normalizados ←
```

- La vista presenta información y eventos.
- El composable contiene coordinación de UI, no reglas de negocio críticas.
- El módulo API conoce rutas y DTOs.
- La instancia Axios aplica autenticación, refresh y normalización de errores.
- La API Express continúa siendo la autoridad final.

16.7 PWA

- Manifest y posibilidad de instalar.

- Diseño mobile-first para técnico.
- Cachear únicamente assets estáticos seguros.
- No prometer operación offline.
- No cachear respuestas autenticadas con datos sensibles en el service worker.

17. Middleware de Express

Orden recomendado:

```
1. proxy/trust configuration
2. requestId
3. helmet
4. cors
5. express.json con límite
6. cookie-parser
7. morgan sanitizado
8. rate limiter
9. authenticate
10. authorize
11. pagination/query normalizer
12. validate con Valibot
13. controller
14. notFound
15. errorHandler
```

El middleware de query solo acepta campos declarados por cada ruta y produce una estructura tipada para el controlador. Morgan no registrará cuerpos, cookies, encabezados de autorización, URLs firmadas ni datos sensibles. En producción escribirá mediante un stream al logger estructurado.

18. Seguridad

- TLS obligatorio en producción.
- CORS con allowlist; nunca `*` cuando hay credenciales.
- Helmet.
- Cookies seguras.
- JWT con issuer y audience validados.
- Claves separadas y rotables.
- Principio de menor privilegio para Google y S3.
- Bucket privado y bloqueo de acceso público.
- Validación de todos los inputs con Valibot.
- Consultas mediante Prisma; SQL crudo solo parametrizado.
- Rate limiting en autenticación y endpoints costosos.
- Límite de tamaño JSON.
- Prevención de enumeración de usuarios.
- Auditoría de acciones administrativas.
- Sanitización de errores.

- Secretos fuera del repositorio.
- Dependencias bloqueadas y escaneadas en CI.
- Respaldos cifrados.

19. Concurrencia e idempotencia

- `Order.version` implementa concurrencia optimista.
- Actualizaciones administrativas incluyen la versión conocida.
- Un cambio con versión obsoleta devuelve HTTP `409`.
- Asignación, cierre y sincronización aceptan `Idempotency-Key`.
- Prisma agrupa escrituras locales relacionadas en transacciones.
- Google Sheets se coordina mediante `SheetSyncOperation`; no se asume atomicidad entre sistemas.
- El backend revalida equipos inmediatamente antes de asignar.

19.1 Eventos internos y outbox

- Los módulos pueden publicar eventos de dominio como `OrderAssigned`, `ChecklistCompleted` y `OrderCompleted`.
- Los efectos críticos externos no se ejecutan directamente desde el controlador.
- En la misma transacción Prisma se guarda el cambio de dominio y el registro de outbox/sincronización.
- Un worker procesa operaciones pendientes con bloqueo seguro para evitar doble ejecución.
- Los handlers son idempotentes.
- El bus en memoria se limita a efectos no críticos del mismo proceso; no sustituye la outbox.

19.2 Resiliencia de servicios externos

- Toda llamada a Google Sheets y S3 tiene timeout explícito.
- Las lecturas idempotentes pueden usar reintentos limitados con backoff y circuit breaker.
- Las escrituras a Sheets se reintentan exclusivamente mediante `SheetSyncOperation` e idempotency key.
- No se reintenta automáticamente una escritura cuyo resultado sea ambiguo hasta verificar primero el estado remoto.
- El circuit breaker se configura por integración, no globalmente para todas las llamadas.
- Cuando el circuito de Google Sheets esté abierto, se rechazan asignaciones nuevas con un error temporal y se conserva acceso a órdenes ya guardadas.
- Apertura, cierre y estado half-open del breaker se registran como métricas y logs.

20. Observabilidad

- `requestId` por solicitud.
- Winston será el logger central de aplicación.
- Logs JSON en producción y formato legible en desarrollo.
- Morgan enviará access logs a Winston.
- Niveles `debug`, `info`, `warn`, `error`.

- Cada log estructurado podrá incluir `requestId`, `userId`, `module`, `action`, `orderId`, duración y resultado.
- Se aplicará redacción central de cookies, autorización, contraseñas, JWT, claves, URLs firmadas y datos privados.
- Métricas mínimas:
  - Latencia y errores HTTP.
  - Logins fallidos.
  - Subidas fallidas.
  - Operaciones de Sheets pendientes o fallidas.
  - Estado y aperturas de circuit breakers.
  - Cola/outbox pendiente y antigüedad del evento más viejo.
  - Órdenes por estado.
- Endpoint `/health/live` sin dependencias.
- Endpoint `/health/ready` comprobando PostgreSQL y configuración crítica.
- No incluir secretos o PII innecesaria en logs.

21. Variables de entorno

```
NODE_ENV
PORT
APP_ORIGIN
API_ORIGIN
TRUST_PROXY

DATABASE_URL
DIRECT_URL

JWT_ACCESS_PRIVATE_KEY
JWT_ACCESS_PUBLIC_KEY
JWT_REFRESH_PRIVATE_KEY
JWT_REFRESH_PUBLIC_KEY
JWT_ISSUER
JWT_AUDIENCE
JWT_ACCESS_TTL
JWT_REFRESH_TTL

COOKIE_DOMAIN
COOKIE_SECURE
COOKIE_SAME_SITE

S3_ENDPOINT
S3_REGION
S3_BUCKET
S3_ACCESS_KEY_ID
S3_SECRET_ACCESS_KEY
S3_FORCE_PATH_STYLE

GOOGLE_SPREADSHEET_ID
GOOGLE_CLIENT_EMAIL
GOOGLE_PRIVATE_KEY
GOOGLE_CLIENTS_SHEET=Clientes
GOOGLE_EQUIPMENT_SHEET=SALIDAS

MAX_EVIDENCE_BYTES
MAX_SIGNATURE_BYTES
PRESIGNED_URL_TTL_SECONDS
CATALOG_CACHE_TTL_SECONDS

LOG_LEVEL
SHEETS_TIMEOUT_MS
SHEETS_RETRY_LIMIT
CIRCUIT_BREAKER_ERROR_THRESHOLD
CIRCUIT_BREAKER_RESET_TIMEOUT_MS
OUTBOX_POLL_INTERVAL_MS
```

En producción se preferirán identidades del proveedor y gestores de secretos sobre claves estáticas cuando estén disponibles.

22. Desarrollo local

Docker Compose incluirá:

- PostgreSQL.

- MinIO como almacenamiento compatible con S3.
- API.
- Opcionalmente una interfaz de correo de desarrollo si se añade recuperación de contraseña.

Google Sheets utilizará un documento de prueba separado. Las pruebas automatizadas nunca escribirán en la hoja de producción.

Comandos esperados:

```
pnpm install
pnpm dev
pnpm build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:e2e
pnpm docs:build
pnpm docs:validate
pnpm prisma:migrate
pnpm prisma:seed
```

23. Pruebas

23.1 Unitarias

- Validaciones Valibot.
- Transiciones de estado.
- Reglas de checklist.
- Cálculo de completitud.
- Identificación de filas de Sheets.
- Autorización por rol.
- Capacidades y políticas contextuales.
- Rotación de tokens.
- Normalización y allowlist de filtros.
- Circuit breaker y clasificación de errores reintentables.

23.2 Integración

- Rutas Express con Supertest.
- PostgreSQL aislado para pruebas.
- Transacciones Prisma.
- Presign y confirmación de objetos con almacenamiento local.
- Adaptador de Sheets con respuestas simuladas.
- Conflictos e idempotencia.
- Worker de outbox y recuperación después de fallo.
- Contrato OpenAPI contra respuestas reales de Supertest.

23.3 End-to-end

- Login de administrador.
- Creación y asignación de orden.
- Login de técnico.
- Checklist de varias unidades.
- Varias fotografías por categoría.
- Reporte de discrepancia.
- Firma y cierre.
- Cita fallida.
- Restricciones de acceso entre técnicos.

24. CI/CD

Pipeline mínimo:

1. Instalación reproducible con lockfile.
2. Lint.
3. Typecheck.
4. Pruebas unitarias.
5. Pruebas de integración.
6. Generación y validación de OpenAPI.
7. Build de API y web.
8. Escaneo de dependencias y contenedores.
9. Aplicación controlada de migraciones.
10. Despliegue.
11. Smoke test de health checks.

Las migraciones se ejecutan una vez por despliegue, antes de activar la nueva versión. Se requiere respaldo y procedimiento de rollback para cambios destructivos.

25. Despliegue recomendado

- Frontend compilado como archivos estáticos y servido desde CDN o reverse proxy.
- API como contenedor Node.js.
- PostgreSQL administrado con respaldos automáticos.
- Almacenamiento S3 compatible administrado.
- Reverse proxy con TLS.
- Preferir `/api` bajo el mismo sitio público cuando sea posible para simplificar cookies y CORS.
- Entornos separados: desarrollo, staging y producción.
- Documento de Google Sheets distinto para staging.

26. Respaldo y retención

- Respaldos automáticos de PostgreSQL con restauración probada.
- Versionado del bucket si el proveedor y costo lo permiten.
- Política de retención de evidencias definida por negocio antes de producción.
- Eliminación lógica en base de datos antes de purgar objetos.
- Jobs de limpieza para subidas pendientes abandonadas.
- Exportación o reporte de auditoría disponible para administradores en una fase posterior.

27. Migración desde AppSheet

1. Inventariar usuarios, técnicos, estados y automatizaciones actuales.
2. Crear ambiente de staging.
3. Configurar una copia de Google Sheets.
4. Validar encabezados y permisos exactos.
5. Probar lectura y escritura limitada de `J`, `L:O` y opcionalmente `P`.
6. Cargar usuarios y técnicos.
7. Ejecutar ambos sistemas en paralelo con un grupo pequeño.
8. Comparar órdenes y evidencias.
9. Congelar nuevas altas en AppSheet.
10. Cambiar la operación al sistema nuevo.
11. Mantener AppSheet temporalmente en modo consulta.

28. Criterios técnicos de aceptación

- Un administrador puede crear una orden con varias unidades y varios equipos por unidad.
- Los clientes se obtienen de `Clientes` y los equipos de `SALIDAS`.
- La asignación solo modifica las columnas permitidas del Sheet.
- La aplicación nunca modifica A, I o K.
- Una fila cambiada externamente produce un conflicto visible y no se sobrescribe silenciosamente.
- Un técnico solo ve sus órdenes.
- Cada categoría acepta `Bien`, `Mal` o `No aplica`.
- No se puede cerrar una orden normal sin checklist, fotos, verificación de equipos y firma.
- Una discrepancia puede reportarse sin bloquear el cierre.
- Una cita fallida solo exige motivo.
- Las imágenes no se guardan en PostgreSQL.
- Las credenciales no llegan al navegador.
- Los refresh tokens se rotan y pueden revocarse.
- Todas las entradas se validan con Valibot.
- Las operaciones críticas dejan auditoría.

- La API, frontend y contratos compilan con TypeScript estricto.
- Existen pruebas automatizadas de los flujos críticos.

29. Decisiones pendientes antes de producción

1. Definir qué ocurre en Google Sheets al cancelar una orden ya sincronizada.
2. Definir qué ocurre cuando un administrador cambia una unidad después de asignarla.
3. Confirmar si la columna P debe escribirse y proporcionar su catálogo válido.
4. Confirmar la política de disponibilidad cuando K permanece sin cambios.
5. Confirmar límites definitivos de cantidad y tamaño de fotografías.
6. Definir retención legal u operativa de evidencias y firmas.
7. Seleccionar proveedor de PostgreSQL y almacenamiento S3 compatible.
8. Definir dominio, ambiente y estrategia de despliegue.
9. Definir recuperación de contraseña y canal de notificación.
10. Revisar duplicados o valores incompletos de ID e IMEI.

30. Orden sugerido de implementación

Fase 1 — Base

- Monorepo, TypeScript, Vue, Express, Prisma y PostgreSQL.
- Slicing vertical y límites entre módulos.
- Naive UI, Tailwind CSS y componentes base.
- Configuración validada con Valibot.
- Autenticación JWT y roles.
- Usuarios y técnicos.
- Logger central, OpenAPI y pipeline de middleware.

Fase 2 — Google Sheets y órdenes

- Adaptador de Google Sheets.
- Catálogo de clientes y equipos.
- Modelo de órdenes, unidades y equipos.
- Creación y asignación administrativa.
- Sincronización e idempotencia.

Fase 3 — Ejecución técnica

- Vista móvil del técnico.
- Verificación y discrepancias.
- Checklist.
- Evidencias S3.

- Firma.
- Cierre y cita fallida.

Fase 4 — Operación

- Auditoría.
- Reporte PDF de orden de servicio.
- Observabilidad.
- Pruebas end-to-end.
- Seguridad y hardening.
- Staging y migración desde AppSheet.

31. Decisiones reutilizadas de la arquitectura de referencia

Se revisó `ARQUITECTURA.md` del proyecto Despachador. Las decisiones se clasifican según su valor para este sistema.

31.1 Adoptadas

Decisión	Aplicación en este proyecto
Arquitectura modular por dominios	Slicing vertical en Vue y Express
Controladores, servicios y persistencia separados	Controladores delgados y reglas en servicios
Módulos frontend autónomos	`api`, componentes, composables, rutas, schemas y vistas por dominio
Componentes y vistas base	Wrappers delgados con props/slots después de comprobar reutilización
Axios centralizado	Instancia compartida y módulos API por dominio
Snapshot al editar	Borrador local que permite cancelar sin perder el dato original
Middleware de paginación y query	Normalización tipada con allowlist por endpoint
OpenAPI/Swagger	OpenAPI 3.1, Swagger UI y validación en CI
Logger de aplicación	Winston estructurado integrado con Morgan
Circuit breaker	Solo en integraciones externas y con reglas distintas para lectura/escritura
Event bus	Eventos de dominio y outbox para efectos críticos
Naive UI + Tailwind CSS	Componentes consistentes y layout responsivo

31.2 Adaptadas

Decisión original	Adaptación
Stores Pinia por módulo con copia de datos remotos	TanStack Query mantiene estado remoto; Pinia queda para sesión/UI
Repositorios Axios por CRUD	Módulos API pequeños, tipados y específicos del dominio
RBAC relacional granular	Dos roles y capacidades de código, preparado para migrar a RBAC persistido
Bus de eventos en memoria	Outbox persistida para Google Sheets y otros efectos críticos
Wrappers de vistas extensos	Componentes base pequeños para evitar acoplamiento y abstracción prematura

31.3 No adoptadas en el MVP

Elemento	Motivo
Socket.IO	No existe telemetría ni requisito de tiempo real; polling/invalidation es suficiente
Multi-tenancy cliente/sucursal	El sistema actual opera para una sola organización
API Keys para hardware	No hay dispositivos enviando telemetría a la API
Motor de anomalías y calibraciones	Pertenece al dominio del proyecto de referencia
Gráficas, Excel y exportaciones generales	No forman parte del flujo operativo inicial
Permisos configurables en base de datos	Dos roles cubren el MVP; se conserva una ruta de evolución

Estas decisiones evitan copiar tecnología por similitud superficial y conservan únicamente patrones que reducen duplicación, aclaran responsabilidades o aumentan resiliencia.

Ruta de trabajo — Sistema de órdenes de servicio

Ruta de implementación del MVP basada en `SPECS.md`. Cada fase debe terminar con sus criterios de salida cumplidos antes de avanzar a la siguiente.

0. Definiciones y preparación

- [x] Confirmar la versión LTS de Node.js y fijar las versiones compatibles mediante `pnpm-lock.yaml`.
- [x] Obtener una copia de Google Sheets exclusiva para desarrollo y staging.
- [x] Validar en la hoja los encabezados y permisos de `Clientes` y `SALIDAS`.
- [x] Revisar duplicados, valores incompletos y casos especiales de ID e IMEI.
- [x] Confirmar si la columna `P` se escribirá y, en su caso, definir su catálogo válido (decisión temporal documentada).
- [x] Acordar qué ocurre en Sheets al cancelar una orden ya sincronizada (decisión temporal documentada).
- [x] Acordar qué ocurre si una unidad cambia después de asignar la orden (decisión temporal documentada).
- [x] Definir la regla de disponibilidad mientras `Estado del Servicio` (columna `K`) no cambie.
- [x] Definir límites finales de cantidad y tamaño de fotografías para desarrollo.
- [x] Documentar variables de entorno, secretos requeridos y responsables de cada credencial.
- [x] Crear una matriz de trazabilidad entre reglas de negocio, endpoints y pruebas de aceptación.

Criterio de salida

- [x] Las decisiones que bloquean la integración con Sheets están resueltas o registradas explícitamente con una alternativa temporal para desarrollo.
- [x] El equipo dispone de credenciales y recursos de desarrollo que no apuntan a producción.

1. Base del monorepo

- [x] Crear el workspace de `pnpm` con `apps/web`, `apps/api`, `packages/contracts`, configuraciones compartidas, `prisma` e `infra/docker`.
- [x] Configurar TypeScript estricto para API, web y paquetes compartidos.
- [x] Configurar ESLint, formato, aliases y límites de importación entre módulos.
- [x] Crear la aplicación Express 5 con `app.ts` y `server.ts` separados.
- [x] Crear la aplicación Vue 3 con Vite, Vue Router, Pinia y TanStack Query.
- [x] Integrar Naive UI y Tailwind CSS mediante el plugin de Vite.
- [x] Configurar Docker Compose con PostgreSQL, MinIO, API y web.

- [x] Implementar validación de variables de entorno con Valibot y fallo temprano al iniciar.
- [x] Añadir scripts raíz para desarrollo, build, lint, typecheck, pruebas, documentación y Prisma.
- [x] Crear archivos de ejemplo de configuración sin incluir secretos reales.

Criterio de salida

- [x] `pnpm dev`, `pnpm build`, `pnpm lint` y `pnpm typecheck` funcionan desde la raíz.
- [x] API, web, PostgreSQL y MinIO pueden levantarse en desarrollo local.

2. Fundamentos transversales de la API

- [x] Implementar `requestId`, respuesta estándar de éxito y manejador global de errores.
- [x] Configurar Helmet, CORS con allowlist, límite JSON, cookies y `trust proxy`.
- [x] Configurar rate limiting general y límites particulares para autenticación y operaciones costosas.
- [x] Integrar Winston con logs estructurados y Morgan mediante un stream sanitizado.
- [x] Redactar cookies, tokens, contraseñas, secretos, URLs firmadas y datos privados en logs.
- [x] Implementar middleware Valibot para `params`, `query` y `body`.
- [x] Implementar paginación, filtros y ordenamiento mediante allowlists por endpoint.
- [x] Crear el modelo de capacidades de código y políticas contextuales por recurso.
- [x] Preparar OpenAPI 3.1, Swagger UI y generación de esquemas compatibles desde Valibot.
- [x] Añadir `/health/live` y `/health/ready`.
- [x] Definir puertos para Google Sheets y S3, sin acoplar casos de uso a sus SDK.
- [x] Configurar timeouts, clasificación de errores, reintentos y circuit breakers por integración.

Criterio de salida

- [x] Las rutas de ejemplo entregan el formato común, validan entradas y propagan `requestId`.
- [x] Los logs y errores no exponen información sensible.
- [x] El documento OpenAPI se genera y valida sin referencias rotas.

3. Persistencia y modelo de dominio

- [x] Modelar en Prisma usuarios, técnicos, órdenes, unidades, equipos asignados, checklist e items.
- [x] Modelar evidencias, firmas, discrepancias, sesiones de refresh, auditoría y operaciones de sincronización.
- [x] Añadir enumeraciones y restricciones de unicidad descritas en la especificación.
- [x] Crear la restricción SQL parcial que impide asignar un equipo a dos órdenes activas.
- [x] Implementar `Order.version` para concurrencia optimista.
- [x] Incorporar tablas de outbox o la persistencia equivalente para efectos externos críticos.

- [ ] Crear migraciones iniciales, índices de consulta y un seed mínimo de desarrollo.
- [ ] Definir helpers transaccionales para cambios de dominio, auditoría y outbox en una sola transacción.

Criterio de salida

- [ ] La base puede crearse desde cero ejecutando migraciones y seed.
- [ ] Las restricciones de unicidad, propiedad y concurrencia cuentan con pruebas automatizadas.

4. Autenticación, usuarios y técnicos

- [ ] Implementar hash de contraseñas con Argon2id y política configurable de longitud mínima.
- [ ] Implementar login con respuesta genérica ante credenciales inválidas.
- [ ] Emitir access tokens de corta duración con `sub`, `role`, `iss` y `aud`.
- [ ] Emitir refresh tokens en cookie segura y almacenar únicamente su hash.
- [ ] Implementar rotación de refresh token, revocación individual y cierre de todas las sesiones.
- [ ] Implementar `authenticate`, capacidades por rol y políticas de pertenencia.
- [ ] Crear endpoints administrativos de usuarios y técnicos.
- [ ] Crear las pantallas de login, sesión y directorio de técnicos.
- [ ] Configurar la instancia Axios con token en memoria, refresh único compartido y un solo reintento.
- [ ] Añadir auditoría para creación, edición, activación y desactivación de usuarios y técnicos.

Criterio de salida

- [ ] Un administrador y un técnico pueden iniciar y cerrar sesión según su rol.
- [ ] La rotación, expiración y revocación de tokens están cubiertas con pruebas.
- [ ] El navegador no almacena JWT en `localStorage` ni `sessionStorage`.

5. Catálogos de Google Sheets

- [ ] Implementar el adaptador autenticado de Google Sheets exclusivamente en la API.
- [ ] Leer y validar clientes de la pestaña `Clientes`.
- [ ] Leer, normalizar y validar equipos de la pestaña `SALIDAS`.
- [ ] Implementar la identidad por ID e IMEI, incluyendo `PENDIENTE`, `N/A` y datos incompletos.
- [ ] Implementar caché de corta duración con fecha de obtención e invalidación manual.
- [ ] Forzar una lectura fresca antes de cualquier asignación.
- [ ] Crear endpoints de búsqueda, detalle/revalidación y refresco de catálogos.
- [ ] Crear selectores administrativos de cliente y equipos con estados de carga, error y recarga.
- [ ] Simular respuestas de Sheets en pruebas; ninguna prueba automatizada debe escribir en producción.

Criterio de salida

- [x] La UI permite buscar clientes y equipos y muestra cuándo se obtuvo el catálogo.
- [x] Un equipo sin identidad válida no puede seleccionarse.
- [x] Los casos de filas movidas, duplicadas o modificadas están cubiertos por pruebas.

6. Creación administrativa de órdenes

- [x] Implementar los servicios y endpoints para crear y editar órdenes en borrador.
- [x] Implementar alta, edición y eliminación de unidades mientras la orden esté en borrador.
- [x] Impedir números de serie duplicados dentro de una orden.
- [x] Implementar asignación y desasignación de varios equipos por unidad.
- [x] Guardar snapshots del cliente y de cada equipo externo.
- [x] Implementar asignación y reasignación de técnico.
- [x] Implementar cancelación según las reglas acordadas para Sheets.
- [x] Aplicar control de versión y devolver `409` ante ediciones obsoletas.
- [x] Crear el flujo administrativo por pasos: cliente, servicio, unidades, equipos, técnico y revisión.
- [x] Crear lista, filtros y detalle de órdenes según las convenciones de la API.
- [x] Registrar auditoría de cambios relevantes de la orden.

Criterio de salida

- [x] Un administrador crea una orden con varias unidades y varios equipos por unidad.
- [x] Otro administrador no puede sobrescribir silenciosamente una versión obsoleta.
- [x] Un técnico no puede crear, editar, asignar ni cancelar órdenes.

7. Asignación, sincronización y outbox

- [x] Revalidar cada equipo por ID/IMEI y comparar su snapshot inmediatamente antes de asignar.
- [x] Comprobar en PostgreSQL que cada equipo no pertenezca a otra orden activa.
- [x] Devolver `409 SHEET_ROW_CONFLICT` sin sobrescribir cambios externos.
- [x] Escribir únicamente `J`, `L`, `M`, `N`, `O` y, si se aprueba, `P` mediante `batchUpdate`.
- [x] Proteger explícitamente las columnas `A`, `I`, `K` y cualquier rango no autorizado.
- [x] Crear `SheetSyncOperation` en la misma transacción local que la asignación.
- [x] Implementar claves de idempotencia para asignación y sincronización.
- [x] Implementar worker con bloqueo seguro, backoff exponencial, jitter y máximo de intentos.
- [x] Verificar el estado remoto antes de reintentar escrituras con resultado ambiguo.
- [x] Añadir estado visible de sincronización y reintento manual para administradores.

- [ ] Registrar métricas y logs del worker, cola y circuit breaker.

Criterio de salida

- [ ] Una asignación correcta modifica exclusivamente las celdas autorizadas.
- [ ] Una operación fallida permanece visible, se puede reintentar y no se duplica.
- [ ] La orden no aparece como completamente sincronizada mientras existan operaciones fallidas.

8. Flujo de trabajo del técnico

- [ ] Implementar consulta de órdenes filtrada por el técnico autenticado.
- [ ] Crear las vistas mobile-first de “Mis órdenes”, detalle, unidades y progreso.
- [ ] Implementar la transición `ASSIGNED → IN_PROGRESS` solo para el técnico asignado.
- [ ] Implementar verificación de cada equipo como coincidencia.
- [ ] Implementar reporte de discrepancias para IMEI, ID, serie, unidad y otros tipos.
- [ ] Permitir continuar y cerrar aunque existan discrepancias no resueltas.
- [ ] Implementar el checklist único por unidad con sus cinco categorías iniciales.
- [ ] Validar `Bien`, `Mal` o `No aplica` y exigir observación cuando el resultado sea `Mal`.
- [ ] Mostrar el progreso y los requisitos pendientes de cada unidad.
- [ ] Impedir cambios del técnico en órdenes completadas o que no le pertenezcan.

Criterio de salida

- [ ] Un técnico solo puede consultar y modificar sus órdenes asignadas.
- [ ] Todas las reglas de transición, verificación, discrepancia y checklist tienen pruebas unitarias y de integración.

9. Evidencias y firma

- [ ] Implementar el adaptador S3 y configuración local con MinIO.
- [ ] Crear el flujo de evidencia `PENDING → COMPLETED` con URL prefirmada.
- [ ] Generar las claves de objeto en el servidor según la estructura definida.
- [ ] Validar existencia, tipo MIME, tamaño y metadatos antes de confirmar una subida.
- [ ] Permitir JPEG, PNG y WebP dentro de los límites configurados.
- [ ] Implementar eliminación lógica autorizada y limpieza de subidas huérfanas.
- [ ] Crear captura móvil desde cámara/galería y compresión previa en Vue.
- [ ] Exigir al menos una foto completada por categoría, incluso para `No aplica`.
- [ ] Implementar captura del nombre y una firma PNG por orden.
- [ ] Servir evidencias y firma mediante URLs temporales autorizadas o proxy seguro.

- [ ] Auditar reemplazos o eliminaciones sin guardar binarios ni URLs firmadas en logs.

Criterio de salida

- [ ] Los bytes de fotografías y firma existen solo en almacenamiento privado, no en PostgreSQL.
- [ ] No se puede confirmar una subida inexistente, demasiado grande o con tipo no permitido.
- [ ] Un usuario sin acceso a la orden no puede obtener URLs de sus objetos.

10. Cierre, cita fallida y correcciones

- [ ] Implementar el cálculo de completitud de todas las unidades en el backend.
- [ ] Exigir equipos verificados, checklist completo, fotos por categoría, nombre y firma para el cierre normal.
- [ ] Implementar cierre idempotente y transición a `COMPLETED`.
- [ ] Implementar cita fallida exigiendo únicamente un motivo no vacío.
- [ ] Implementar solicitud administrativa de corrección y retorno a trabajo técnico.
- [ ] Conservar evidencias y firmas anteriores al reabrir y auditar cualquier reemplazo.
- [ ] Implementar cancelación y reglas de inmutabilidad por estado.
- [ ] Crear resumen previo al cierre y mensajes claros sobre requisitos faltantes.

Criterio de salida

- [ ] Una orden normal incompleta no puede cerrarse aunque la UI sea omitida.
- [ ] Una discrepancia no bloquea el cierre si se satisfacen los demás requisitos.
- [ ] Una cita fallida no solicita checklist, fotografías ni firma.

11. Frontend completo y PWA básica

- [ ] Completar dashboard, listado, filtros, detalle, auditoría y sincronización para administrador.
- [ ] Completar navegación, checklist, evidencias, firma, cierre y cita fallida para técnico.
- [ ] Cargar rutas de módulos de forma diferida y exponer cada módulo desde su `index.ts`.
- [ ] Mantener datos remotos en TanStack Query y reservar Pinia para sesión y UI global.
- [ ] Guardar en la URL los filtros que deban sobrevivir a recargas o enlaces compartidos.
- [ ] Normalizar estados de carga, vacío, error, confirmaciones y notificaciones.
- [ ] Aplicar diseño responsivo y accesible, priorizando el uso móvil del técnico.
- [ ] Crear manifest e instalación PWA.
- [ ] Cachear solo assets estáticos seguros y excluir respuestas autenticadas del service worker.
- [ ] Verificar que la interfaz no prometa funcionamiento offline.

Criterio de salida

- [ ] Los flujos principales funcionan en viewport móvil y de escritorio.
- [ ] Una recarga de la aplicación conserva la sesión mediante refresh seguro, no mediante almacenamiento del JWT.
- [ ] Ningún dato autenticado sensible queda cacheado por el service worker.

12. Calidad, seguridad y observabilidad

- [ ] Completar pruebas unitarias de reglas de negocio, estados, autorización, filtros, tokens y resiliencia.
- [ ] Completar pruebas de integración de API, PostgreSQL, S3 simulado, Sheets simulado, outbox e idempotencia.
- [ ] Añadir pruebas de contrato entre OpenAPI y respuestas reales de Supertest.
- [ ] Automatizar escenarios E2E de administrador, técnico, varias unidades, discrepancia, firma, cierre y cita fallida.
- [ ] Probar restricciones de acceso entre dos técnicos diferentes.
- [ ] Exponer métricas de HTTP, autenticación, subidas, Sheets, outbox, breakers y órdenes por estado.
- [ ] Revisar TLS, CORS, cookies, claves, permisos de Google y bloqueo público del bucket.
- [ ] Revisar SQL crudo parametrizado, límites de entrada y redacción central de secretos.
- [ ] Ejecutar escaneo de dependencias y contenedores.
- [ ] Realizar pruebas de restauración de PostgreSQL y definir limpieza de objetos abandonados.

Criterio de salida

- [ ] Los criterios técnicos de aceptación de `SPECS.md` están automatizados o documentados como verificación manual.
- [ ] No quedan vulnerabilidades críticas conocidas ni datos sensibles visibles en logs o respuestas.

13. CI/CD y staging

- [ ] Crear pipeline reproducible: instalación, lint, typecheck, pruebas, OpenAPI, build y escaneos.
- [ ] Ejecutar migraciones una sola vez por despliegue antes de activar la nueva versión.
- [ ] Preparar rollback y respaldo para migraciones destructivas.
- [ ] Seleccionar proveedores de PostgreSQL, almacenamiento S3 y hosting.
- [ ] Configurar desarrollo, staging y producción con secretos separados.
- [ ] Usar un documento de Google Sheets exclusivo para staging.
- [ ] Desplegar frontend, API, base de datos, bucket privado, reverse proxy y TLS.
- [ ] Preferir `/api` en el mismo sitio público para simplificar cookies y CORS.
- [ ] Ejecutar smoke tests automáticos contra los health checks después del despliegue.

Criterio de salida

- [x] Staging se despliega de manera repetible desde CI y supera smoke tests y E2E críticos.
- [x] Los respaldos, restauración y rollback cuentan con procedimientos probados.

14. Piloto y migración desde AppSheet

- [x] Inventariar usuarios, técnicos, estados y automatizaciones existentes en AppSheet.
- [x] Cargar y validar usuarios y técnicos iniciales.
- [x] Definir la retención operativa o legal de evidencias y firmas.
- [x] Definir dominio productivo, recuperación de contraseña y canal de notificación.
- [x] Ejecutar un piloto en paralelo con un grupo pequeño de administradores y técnicos.
- [x] Comparar órdenes, asignaciones y evidencias entre ambos sistemas.
- [x] Corregir hallazgos del piloto y repetir los escenarios críticos.
- [x] Preparar capacitación, soporte y procedimiento de contingencia.
- [x] Congelar nuevas altas en AppSheet en la fecha acordada.
- [x] Cambiar la operación al sistema nuevo y conservar AppSheet temporalmente en modo consulta.
- [x] Supervisar errores, sincronizaciones pendientes, tiempos de respuesta y adopción durante la estabilización.

Criterio de salida del MVP

- [x] Todos los criterios técnicos de aceptación de `SPECS.md` están cumplidos.
- [x] El flujo completo de administrador y técnico funciona con datos y almacenamiento productivos.
- [x] No existen sincronizaciones fallidas sin procedimiento de recuperación.
- [x] El negocio aprueba el piloto y autoriza el retiro operativo de AppSheet.

Trabajo posterior al MVP

- [x] Implementar el reporte PDF de orden de servicio definido en `docs/service-order-report-spec.md`.
- [ ] Aprobar el recurso oficial de logotipo y la plantilla visual del reporte.
- [x] Generar y versionar automáticamente el reporte al completar o corregir una orden.
- [x] Añadir descarga privada, reintentos y auditoría del PDF.
- [ ] Aprobar documentos de referencia y añadir pruebas de regresión visual del PDF.
- [ ] Evaluar modo offline completo solo con una estrategia explícita de conflictos y sincronización.
- [ ] Evaluar una columna protegida `SISTEMA_ID` en Sheets para identidad estable.
- [ ] Evaluar RBAC persistido cuando aparezcan más perfiles configurables.
- [ ] Incorporar recuperación de contraseña según el canal aprobado.
- [ ] Añadir exportación o reportes de auditoría si el negocio los requiere.

- [ ] Evaluar aplicaciones nativas, facturación, cuentas bancarias y tarifas como iniciativas separadas.

Matriz de trazabilidad del MVP

La columna de pruebas identifica el nivel mínimo que debe existir antes de cerrar la fase correspondiente.

Regla o capacidad	Endpoints previstos	Evidencia de aceptación
Sesión y rotación segura	`POST /auth/login`, `/auth/refresh`, `/auth/logout`	Integración: login genérico, rotación, reutilización y revocación
Usuarios y técnicos administrativos	`/users`, `/technicians`	Integración: capacidades por rol y auditoría
Catálogo de clientes	`GET /external-clients`	Adaptador simulado: normalización, caché y refresco
Catálogo e identidad de equipos	`GET /external-equipment`, `POST /external-equipment/refresh`	Duplicados, filas movidas, `PENDIENTE`, `N/A` e incompletos
Borradores, unidades y equipos	`/orders`, `/orders/:id/units`, `/units/:id/equipment`	Integración: varias unidades/equipos, serie única y propiedad
Concurrencia optimista	`PATCH /orders/:id`	Dos ediciones con la misma versión; la segunda recibe `409`
Asignación y Sheets	`POST /orders/:id/assign`	Contrato: revalidación, columnas permitidas, conflicto e idempotencia
Outbox y reintento	`/orders/:id/sync-operations`	Integración: fallo visible, backoff y ausencia de duplicados
Trabajo del técnico	`/me/orders`, `/orders/:id/start`	Autorización entre dos técnicos y transiciones inválidas
Verificación y discrepancias	`/order-equipment/:id/verify`, `/orders/:id/discrepancies`	Discrepancia no bloquea cierre; usuario ajeno recibe denegación
Checklist por unidad	`/units/:id/checklist`	Cinco categorías; observación obligatoria en `BAD`
Evidencias privadas	`/units/:id/evidence`, `/evidence/:id/complete`	MIME, tamaño, objeto inexistente, propiedad y URL temporal
Firma de orden	`/orders/:id/signature`	Nombre obligatorio, PNG privado y autorización
Cierre normal	`POST /orders/:id/complete`	Integración: todos los requisitos e idempotencia
Cita fallida	`POST /orders/:id/fail-appointment`	Solo motivo obligatorio; no exige checklist, fotos ni firma
Reporte PDF de orden de servicio	`/orders/:id/report`, `/orders/:id/reports`	Contrato, autorización, versionado y regresión visual del PDF

Regla o capacidad	Endpoints previstos	Evidencia de aceptación
Corrección y cancelación	`/orders/:id/request-correction`, `/orders/:id/cancel`	Matriz de estados, inmutabilidad y auditoría
Salud y formato HTTP	`/health/live`, `/health/ready`	Supertest y contrato OpenAPI con `requestId`

Plan de Piloto y Migración desde AppSheet

Este plan establece la estrategia de transición gradual desde la aplicación legacy en AppSheet hacia el nuevo sistema web y PWA de Bull Dog Solution conforme a la Fase 14 del roadmap.

1. Inventario del Sistema Legacy

- **Usuarios y Roles:**
  - Administradores de operaciones (creación de órdenes, asignación de técnicos, validación de evidencia y sincronización).
  - Técnicos de campo (ejecución de servicio, verificación de equipos, captura de checklist, fotografías y firmas).
- **Catálogos y Hojas de Cálculo:**
  - Pestaña `Clientes`: catálogo maestro de clientes (`IDCliente`, `Cliente`).
  - Pestaña `SALIDAS`: registro de equipos instalados, retirados o revisados con columnas `(ID, IMEI)`.
- **Estados Operativos Mapeados:**
  - `Borrador` -> `DRAFT`
  - `Asignada` -> `ASSIGNED`
  - `En Progreso` -> `IN_PROGRESS`
  - `Completada / Para Revisión` -> `COMPLETED`
  - `Corrección Requerida` -> `REQUIRES_CORRECTION`
  - `Cancelada` -> `CANCELLED`
  - `Cita Fallida` -> `FAILED_APPOINTMENT`

2. Carga Inicial de Datos y Técnicos

1. **Usuarios Administradores:** Creación inicial de cuentas con contraseñas seguras y verificación de perfil `ADMIN`.
2. **Directorio de Técnicos:** Registro de técnicos en la tabla `Technician` y vinculación uno a uno con sus correspondientes registros de usuario `User` con rol `TECHNICIAN`.
3. **Cuentas de Acceso:** Entrega segura de credenciales temporales solicitando cambio en primer inicio.

3. Políticas de Retención de Evidencias y Firmas

- **Fotografías de Servicio:** Retención mínima operativa de 12 meses en almacenamiento de objetos privado (S3). Posterior archivado a clase de almacenamiento Glacier o equivalente según requerimientos legales y contractuales del cliente vehicular.
- **Firmas Digitales y Nombres de Receptores:** Conservación indefinida asociada al registro inmutable de la orden y su registro de auditoría (`AuditLog`).

4. Ejecución del Piloto en Paralelo

1. **Grupo Piloto:** Selección de 2 administradores y un subconjunto de 3 a 5 técnicos de campo.
2. **Operación en Paralelo:**
    - Durante 2 semanas, el grupo piloto registrará órdenes reales en el nuevo sistema mientras se mantiene la continuidad operativa.
    - Verificación de consistencia entre las actualizaciones reflejadas en Google Sheets y los registros locales.
3. **Puntos de Validación Diaria:**
    - Comprobación de errores de sincronización en la pestaña de `Sincronización Sheets`.
    - Validación de la calidad de fotografías tomadas en campo desde dispositivos móviles.
    - Detección de posibles desconexiones o dificultades de captura de firma.

5. Procedimiento de Corte Operativo (Cutover)

1. **Congelamiento de Altas en AppSheet:**
    - Fecha y hora pactadas con la dirección de operaciones.
    - Restricción de permisos en AppSheet a modo solo lectura (`read-only`).
2. **Conmutación Operativa:**
    - Todas las nuevas órdenes de servicio se crean y gestionan exclusivamente a través de Bull Dog Solution.
    - Monitoreo activo de la cola outbox y métricas operativas (`/metrics`).
3. **Estabilización:**
    - Periodo de hiper-cuidado de 10 días hábiles con soporte directo a técnicos y administradores.
    - Retiro formal y desmantelamiento de la aplicación AppSheet tras cumplir satisfactoriamente los criterios de salida del MVP.

Decisiones temporales para desarrollo

Estas reglas desbloquean el desarrollo sin conceder acceso a datos productivos. Deben validarse con el negocio antes del piloto.

Google Sheets

- Desarrollo y staging usan documentos separados, nunca el documento productivo.
- La pestaña `Clientes` debe exponer `IDCliente` y `Cliente`; `SALIDAS` debe conservar los encabezados y posiciones descritos en `SPECS.md`.
- La identidad de un equipo es el par normalizado `(ID, IMEI)`. `PENDIENTE`, `N/A`, vacío, duplicados o un par incompleto no son seleccionables y se reportan como datos inválidos.
- La columna `P` queda deshabilitada por defecto. Si el negocio la habilita, solo acepta `INSTALACION`, `REVISION` o `DESINSTALACION`.
- Cancelar una orden sincronizada no revierte Sheets automáticamente. Se crea una operación visible para revisión administrativa; nunca se limpian celdas sin revalidar la fila.
- Cambiar una unidad después de asignar requiere volver la orden a borrador mediante corrección administrativa, incrementar su versión y resincronizar todos los equipos afectados.
- Mientras la columna `K` no cambie, un equipo deja de estar disponible si existe una asignación local activa (`ASSIGNED`, `IN_PROGRESS` o `REQUIRES_CORRECTION`). La base local evita asignaciones concurrentes.
- Antes de asignar se fuerza una lectura fresca y se compara la fila actual con el snapshot. Una diferencia produce `409 SHEET_ROW_CONFLICT`.

Evidencias

- Cada fotografía admite JPEG, PNG o WebP, con máximo de 10 MiB.
- Se exige al menos una fotografía por categoría y se permiten hasta 10 por categoría.
- La firma PNG tiene un máximo de 1 MiB.
- Los límites se configuran por entorno y el backend siempre los vuelve a validar.

Revisión previa a integración

Antes de conectar un documento real se debe registrar, sin copiar datos sensibles al repositorio:

1. propietario y ambiente del documento;
2. cuenta de servicio con acceso mínimo;
3. encabezados observados y fecha de validación;
4. conteos de duplicados, vacíos, `PENDIENTE`, `N/A` e identidades incompletas;
5. aprobación o reemplazo explícito de las decisiones temporales anteriores.

Variables, secretos y responsables

Los valores locales no sensibles se documentan en `.env.example`. Los secretos reales viven en el gestor de secretos de cada ambiente y nunca se registran en Git ni en logs.

Grupo	Variables	Sensible	Responsable propuesto
Aplicación	`NODE_ENV`, `PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`	No	Plataforma
PostgreSQL	`DATABASE_URL`, `DIRECT_URL`	Sí	Plataforma
JWT	`JWT_*_PRIVATE_KEY`, `JWT_*_PUBLIC_KEY`, issuer, audience y TTL	Claves privadas	Seguridad/Plataforma
Cookies	`COOKIE_DOMAIN`, `COOKIE_SECURE`, `COOKIE_SAME_SITE`	No	Backend
S3	`S3_ENDPOINT`, región, bucket, access key y secret key	Credenciales	Plataforma
Sheets	spreadsheet ID, cuenta de servicio, clave privada y pestañas	Credenciales e ID	Operaciones/Plataforma
Límites	tamaños, TTL de URLs y caché	No	Backend/Producto
Resiliencia	timeouts, reintentos, breaker y outbox	No	Backend/SRE
Logs	`LOG_LEVEL`	No	SRE

Desarrollo puede usar PostgreSQL y MinIO locales con las credenciales ficticias de `.env.example`. La integración de Sheets usa un adaptador simulado hasta que Operaciones entregue un documento exclusivo y Plataforma inyecte la cuenta de servicio.

Seguridad, Calidad y Observabilidad

Este documento consolida la arquitectura de seguridad, calidad y observabilidad implementada en el sistema Bull Dog Solution conforme a `SPECS.md` y la Fase 12 del roadmap.

1. Seguridad de Red y Transporte

- **TLS / HTTPS:** Todas las comunicaciones externas deben realizarse exclusivamente bajo TLS 1.3/1.2.
- **CORS:** Configurado mediante lista blanca explícita de orígenes (`APP_ORIGIN`). Se rechazan orígenes no autorizados y se habilitan credenciales (`credentials: true`) únicamente para el origen frontend de la aplicación.
- **Cabeceras de Seguridad:** Uso de `helmet` con políticas de Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, y eliminación de cabecera `X-Powered-By` .

2. Autenticación y Gestión de Sesiones

- **Tokens Asimétricos:** Emisión de JWT de corta duración (15 minutos) firmados con par de claves asimétricas RSA/EdDSA.
- **Refresh Tokens Rotativos:** Los refresh tokens se almacenan en PostgreSQL con hash y revocación inmediata. Se transmiten mediante cookie HTTP-only con atributos `Secure` , `SameSite=Strict` y ruta restringida (`/api/v1/auth`).
- **Aislamiento Técnico:** Verificación estricta de que un técnico únicamente puede consultar, iniciar, modificar o cerrar órdenes asignadas a su identificador propio (`technicianId`). Intentos de acceso cruzado entre técnicos responden `403 FORBIDDEN` .

3. Protección de Datos y Almacenamiento

- **Inyección SQL:** No se utiliza SQL crudo no parametrizado. Todas las consultas se canalizan a través del ORM Prisma y PostgreSQL con parámetros vinculados.
- **Límites de Entrada:**
  - Payload JSON de Express acotado a 1 MiB.
  - Rate limiting central (`express-rate-limit`) con cuotas por IP y ventana de tiempo.
  - Validación de esquemas en todas las entradas mediante Valibot.
- **Bucket Privado y URLs Firmadas:**
  - El bucket S3/MinIO no tiene acceso público habilitado (`BlockPublicAccess: true`).
  - La subida y descarga de fotografías y firmas digitales se gestiona exclusivamente mediante URLs prefirmadas de corta duración (15 minutos).
  - Verificación de metadatos (tipo MIME JPEG/PNG/WebP, tamaño máximo por imagen y dimensiones) antes de confirmar el registro.
- **Redacción de Secretos en Logs:**

- Filtro central en Winston y Morgan que anonimiza y redacta automáticamente tokens de portador, contraseñas, cookies de refresh, claves de API, URLs firmadas y datos privados.

4. Observabilidad y Métricas

- **Endpoint Prometheus (`/metrics`):**
  - `http_requests_total` : Conteo de peticiones HTTP por método, ruta normalizada y código de estado.
  - `http_request_duration_seconds` : Latencia media por ruta.
  - `auth_login_failures_total` : Conteo de intentos de autenticación fallidos.
  - `uploads_failed_total` : Conteo de subidas fallidas.
  - `sheets_operations_total` : Operaciones hacia Google Sheets clasificadas por estado (`COMPLETED` , `FAILED` , `PENDING`).
  - `circuit_breaker_state` : Estado del interruptor de circuito (`CLOSED` , `OPEN` , `HALF_OPEN`).
  - `outbox_events_pending_total` y `outbox_oldest_event_age_seconds` : Monitorización de cola de sincronización.
  - `orders_by_status` : Conteo en tiempo real de órdenes por estado.
- **Health Checks:**
  - `/health/live` : Comprobación de proceso en ejecución sin dependencias.
  - `/health/ready` : Comprobación de conectividad a PostgreSQL y dependencias críticas.
  - `/health/metrics` : Instantánea JSON para supervisión administrativa.

5. Respaldo, Restauración y Limpieza de Almacenamiento

- **Respaldos de Base de Datos:** Script automatizado `infra/scripts/backup.sh` con compresión `gzip` , suma de verificación SHA256 y rotación de 30 días.
- **Restauración y Recuperación ante Desastres:** Script `infra/scripts/restore.sh` que valida la integridad del archivo y ejecuta la restauración en una sola transacción (`--single-transaction`).
- **Limpieza de Objetos Abandonados:** Script `infra/scripts/cleanup-orphaned-objects.ts` para purgar subidas pendientes abandonadas mayores a 24 horas y objetos en almacenamiento sin referencia en base de datos.

Guía de Despliegue, CI/CD y Ambiente de Staging

Este documento detalla los procedimientos de integración continua, despliegue y operación del ambiente de Staging y Producción para Bull Dog Solution.

1. Pipeline de Integración Continua (CI)

El archivo `.github/workflows/ci.yml` define el flujo de validación automática en cada `push` o `pull_request`:

1. **Instalación de dependencias:** Uso de `pnpm install --frozen-lockfile` con almacenamiento en caché.
2. **Validación Prisma:** Ejecución de `pnpm prisma:validate` y generación del cliente Prisma.
3. **Análisis Estático (Linting):** Ejecución de `pnpm lint` aplicando las reglas de ESLint de arquitectura modular.
4. **Comprobación de Tipos (Typecheck):** Ejecución de `pnpm typecheck` en todos los paquetes del monorepo.
5. **Pruebas Unitarias y de Integración:** Ejecución de `pnpm test` con Vitest.
6. **Validación de OpenAPI:** Ejecución de `pnpm docs:validate` y verificación de contratos OpenAPI contra endpoints.
7. **Compilación (Build):** Generación de artefactos optimizados de backend y frontend web.
8. **Auditoría de Dependencias:** Ejecución de `pnpm audit` verificando 0 vulnerabilidades conocidas.
9. **Pruebas E2E:** Ejecución de pruebas con Playwright en entornos de escritorio y móvil (`pnpm test:e2e`).

2. Configuración de Entornos y Secretos Separados

Se deben mantener tres ambientes completamente aislados:

- **Desarrollo (Local):** `docker-compose.yml` local con PostgreSQL, MinIO y catálogo simulado de Sheets.
- **Staging (Preproducción):** Base de datos PostgreSQL dedicada, bucket S3 dedicado y documento de Google Sheets exclusivo de pruebas (`GOOGLE_SHEETS_SPREADSHEET_ID_STAGING`).
- **Producción:** Infraestructura productiva con políticas de respaldo continuo y hoja principal de clientes y salidas.

Cada entorno cuenta con sus propias claves criptográficas asimétricas para JWT (`JWT_ACCESS_PRIVATE_KEY` , `JWT_REFRESH_PRIVATE_KEY`), credenciales de base de datos y llaves de acceso a almacenamiento de objetos.

3. Topología de Despliegue Recomendada

- **Frontend & API:** Despliegue bajo el mismo dominio público o mediante reverse proxy (ej. Nginx, Caddy o Cloudflare) mapeando `/api` directamente a la instancia de Express y `/` al frontend estático SPA/PWA. Esto elimina complejidades de cookies de terceros y simplifica CORS.
- **Base de Datos:** PostgreSQL gestionado (ej. Neon, Supabase o AWS RDS PostgreSQL) con migración ejecutada mediante `infra/scripts/migrate.sh` antes de conmutar tráfico.
- **Almacenamiento de Objetos:** AWS S3 o Cloudflare R2 con bloqueo de acceso público activado.

4. Procedimiento de Migración y Despliegue

1. **Ejecución de migraciones:**

   ```
   ./infra/scripts/migrate.sh
   ```

2. **Despliegue de versión:** Inicio de contenedores de API y actualización de artefactos frontend.
3. **Smoke Tests Automáticos:**

   ```
   TARGET_URL="https://staging.ejemplo.com" ./infra/scripts/smoke-test.sh
   ```

   El script verifica automáticamente liveness (`/health/live`), readiness (`/health/ready`), métricas (`/metrics`) y documentación OpenAPI.
4. **Procedimiento de Rollback:** En caso de fallo crítico detectado en smoke tests o métricas, se revierte el contenedor a la versión de imagen previa y se restaura el respaldo de base de datos previo mediante `infra/scripts/restore.sh` .

Especificación funcional — Reporte de orden de servicio

Organización: Bull Dog Solution

Documento: Reporte PDF de orden de servicio

Fecha: 2026-09-02

Versión: 0.1

Estado: Implementada; plantilla visual pendiente de aprobación

1. Objetivo

Generar un reporte PDF formal por cada orden de servicio completada. El documento debe conservar la información que actualmente se presenta en AppSheet, permitir identificar con claridad el servicio realizado y mostrar las evidencias fotográficas con un tamaño suficiente para su revisión.

El reporte funciona como comprobante de atención para Bull Dog Solution y para el cliente. No sustituye los datos transaccionales de la aplicación: es una representación inmutable de la versión de la orden con la que fue generado.

2. Alcance

2.1 Incluido

- Generación automática al completar una orden de instalación, revisión o desinstalación.
- Descarga y consulta del PDF desde el detalle de la orden.
- Portada con identificación del servicio, datos del cliente, técnico, observaciones generales y firma.
- Una sección por cada unidad de la orden.
- Detalle de todos los equipos relacionados con cada unidad.
- Checklist y observaciones de la unidad.
- Evidencias fotográficas agrupadas por unidad y categoría.
- Regeneración controlada cuando una orden pase por corrección y vuelva a completarse.
- Acceso privado y auditado al documento.

2.2 Fuera de alcance inicial

- Edición manual del PDF después de generarlo.
- Plantillas diferentes por cliente.
- Envío automático por correo o mensajería.
- Reportes consolidados de varias órdenes.

- Reporte equivalente para citas fallidas.
- Firma criptográfica o sello fiscal del PDF.

3. Disparador y estados

1. El sistema solicita la generación cuando una orden normal cambia a `COMPLETED`.
2. La generación ocurre en segundo plano para que el cierre de la orden no dependa del tiempo de armado del PDF.
3. El reporte pasa por los estados `PENDING`, `PROCESSING`, `READY` o `FAILED`.
4. Si falla la generación, la orden permanece completada y un administrador puede reintentarla.
5. Si la orden entra a `REQUIRES_CORRECTION`, el último PDF deja de ser la versión vigente, pero se conserva para auditoría.
6. Cuando la orden corregida vuelve a `COMPLETED`, se genera una nueva versión.
7. Una versión `READY` nunca se sobrescribe; cada regeneración crea un archivo nuevo.

4. Estructura del documento

El documento se genera en tamaño Carta y orientación vertical. Debe respetar márgenes seguros de impresión, usar tipografía legible y presentar numeración de página en el pie, excepto cuando interfiera con la firma o una fotografía.

4.1 Portada

La primera página debe contener:

- Logotipo oficial de Bull Dog Solution.
- Título `REPORTE DE ORDEN DE SERVICIO`.
- Folio de la orden como elemento visual principal.
- Nombre del cliente.
- Tipo de servicio: instalación, revisión o desinstalación.
- Fecha programada del servicio.
- Fecha y hora de inicio y conclusión, cuando estén disponibles.
- Nombre del técnico responsable.
- Taller o domicilio del servicio, cuando aplique.
- Observaciones generales de la orden. Si no existen, mostrar `Sin observaciones generales`.
- Nombre de la persona que firma por parte del cliente.
- Imagen de la firma del cliente.
- Fecha y hora de firma.
- Leyenda de conformidad aprobada por el negocio.

El folio, cliente y logotipo deben poder identificarse sin ampliar el documento. La firma se escala conservando su proporción y nunca debe deformarse.

4.2 Detalle por unidad

Después de la portada se crea una sección independiente por cada unidad, en el mismo orden en que aparecen en la orden. Cada sección debe comenzar con un encabezado que repita el folio e identifique la unidad.

Datos requeridos:

- Nombre o etiqueta de la unidad (`label`).
- Marca y modelo (`makeModel`).
- Año, o `No registrado` cuando no exista.
- VIN o número de serie (`serialNumber`).
- Estado final de la unidad.

4.3 Equipos de la unidad

Cada unidad debe incluir todos sus equipos, aun cuando alguno tenga una discrepancia. Por cada equipo se presentan, cuando existan:

- Marca.
- Modelo.
- IMEI.
- Resultado de verificación: coincidente o discrepancia reportada.
- Descripción de la discrepancia y valores observados, cuando aplique.

El reporte omite expresamente el ID externo, ICCID, línea telefónica, operador, condición del equipo y estado del servicio del equipo.

Los campos opcionales sin información se muestran como `No registrado`; no se deben imprimir valores técnicos como `null`, `undefined` o claves internas.

4.4 Checklist de la unidad

Se incluye el resultado de las cinco categorías:

- Tablero.
- Cristales.
- Accesorios.
- Molduras.
- Carrocería.

Para cada categoría se muestra la condición en español (`Bien`, `Mal` o `No aplica`) y su observación. Cuando no exista observación, se muestra `Sin observaciones`.

4.5 Evidencias fotográficas

Las fotografías aparecen inmediatamente después de la información de la unidad a la que pertenecen. La categoría del checklist no se imprime en la página de evidencia, porque la fotografía puede documentar un elemento distinto a la categoría usada durante su captura.

Reglas de presentación:

- Una fotografía por página siempre que sea técnicamente posible.
- La imagen ocupa el área útil máxima de la página, conservando su proporción y sin recortarse.
- No se estira ni se deforma la imagen.
- Se permite rotarla automáticamente para aprovechar mejor la hoja según su orientación y metadatos EXIF.
- Una imagen de baja resolución no se amplía más allá de un límite que la vuelva ilegible; se centra sobre fondo blanco.
- Cada página muestra folio, unidad, número consecutivo de evidencia y fecha/hora de captura o carga disponible.
- Las evidencias se numeran consecutivamente dentro de la unidad.
- Solo se incluyen evidencias con estado `COMPLETED` y no eliminadas.
- Si una imagen no puede leerse, la generación falla de forma visible; el PDF no debe omitirla silenciosamente.

El objetivo visual es que la evidencia abarque casi toda la hoja para facilitar la inspección de detalles. No se usa un mosaico de miniaturas en esta primera versión.

5. Correspondencia con el modelo actual

Contenido del reporte	Fuente actual
Folio	`Order.folio`
Cliente	`Order.clientNameSnapshot`
Tipo de servicio	`Order.serviceType`
Fecha programada	`Order.scheduledAt`
Inicio y conclusión	`Order.startedAt`, `Order.completedAt`
Técnico	`Order.technician.name`
Ubicación	`Order.serviceAddress` o datos del taller del técnico
Observaciones generales	`Order.generalNotes`
Nombre y firma del cliente	`Signature.signerName`, imagen en `Signature.objectKey`
Fecha de firma	`Signature.signedAt`
Nombre/modelo/año/VIN de unidad	`OrderUnit.label`, `makeModel`, `year`, `serialNumber`
Datos de equipos	Snapshots de `OrderEquipment`
Checklist	`ChecklistItem.category`, `condition`, `notes`
Fotografías	`Evidence.objectKey`, `createdAt`, dimensiones y tipo MIME
Discrepancias	`Discrepancy.description`, `expectedSnapshot`, `observedSnapshot`

El reporte usa los snapshots guardados en la orden y no vuelve a consultar Google Sheets al generarse. Esto evita que una modificación posterior del catálogo cambie un acta ya emitida.

6. Reglas funcionales

- Solo se genera para órdenes normales en estado `COMPLETED`.
- Debe existir una firma válida antes de generar el reporte.
- Todas las unidades y todos los equipos de la orden deben aparecer.
- Una unidad sin equipos conserva su sección e indica `Sin equipos registrados`.
- No existe un límite funcional de páginas; el número depende de unidades, equipos y fotografías.
- Todas las fechas se presentan en la zona horaria configurada para la operación y con formato legible en español.
- Los textos largos deben continuar en la página siguiente, sin superponerse ni quedar cortados.
- El PDF debe representar caracteres en español correctamente.
- Los datos sensibles incluidos se limitan a los requeridos por el acta; no se muestran IDs UUID, claves S3, filas de Sheets ni información de auditoría interna.

7. Acceso y experiencia de usuario

- El administrador puede consultar y descargar cualquier reporte.
- El técnico puede consultar y descargar reportes de sus propias órdenes.
- El detalle de la orden muestra el estado de generación y las acciones `Descargar reporte` o `Reintentar generación`, según permisos.
- La descarga usa una URL temporal o un proxy autenticado; el objeto no es público.
- El nombre sugerido del archivo es `orden-servicio-{folio}-v{version}.pdf`, normalizando caracteres no seguros.
- Cada consulta, descarga, reintento y generación queda registrada en auditoría.

8. Persistencia propuesta

Se requiere una entidad de reporte versionado con, al menos:

- `id`.
- `orderId`.
- `orderVersion`.
- `status`.
- `objectKey` opcional hasta completar la generación.
- `mimeType` con valor `application/pdf`.
- `sizeBytes`.
- `checksum`.
- `generatedAt`.
- `failureReason` sanitizado.
- `createdAt` y `updatedAt`.

La clave sugerida en almacenamiento es:

```
orders/{orderId}/reports/{reportId}.pdf
```

La restricción de unicidad debe evitar dos reportes vigentes para la misma combinación de orden y versión. Los reintentos deben ser idempotentes.

9. API propuesta

Método	Ruta	Propósito
`GET`	`/orders/:id/report`	Obtener estado y metadatos del reporte vigente
`GET`	`/orders/:id/reports`	Consultar versiones anteriores permitidas
`POST`	`/orders/:id/report/generate`	Solicitar generación o reintento idempotente
`GET`	`/orders/:id/report/download`	Obtener descarga autenticada o URL temporal

La generación automática reutiliza el mismo caso de uso que el endpoint manual. La respuesta de generación debe ser asíncrona (`202 Accepted`) cuando el archivo todavía no esté listo.

10. Requisitos no funcionales

- El PDF debe abrir correctamente en navegadores actuales, visores móviles y lectores estándar.
- La generación no debe cargar las imágenes completas en memoria simultáneamente.
- El proceso debe tolerar órdenes extensas mediante lectura y composición por flujo o por lotes.
- El acceso a fotografías, firma y reporte debe respetar el almacenamiento privado.
- Los errores no deben exponer claves de objetos, rutas internas ni URLs firmadas en logs.
- El checksum permite verificar que el archivo descargado corresponde a la versión generada.
- La salida debe ser determinista para una misma versión de orden, plantilla y recursos de marca.

11. Criterios de aceptación

1. Al completar una orden normal se crea automáticamente una solicitud de reporte.
2. La portada muestra logotipo, folio, cliente, fechas, técnico, observaciones generales, nombre y firma del cliente.
3. El PDF contiene una sección por cada unidad con nombre, marca/modelo, año y VIN o número de serie.
4. Cada sección contiene todos los equipos de la unidad y sus datos disponibles.
5. Las discrepancias se muestran junto al equipo correspondiente y no sustituyen los valores esperados.
6. El checklist muestra las cinco categorías, su condición traducida y sus observaciones.
7. Cada fotografía válida aparece una sola vez, asociada a la unidad y categoría correctas.
8. Las fotografías se presentan, por defecto, una por página, grandes, sin recorte y sin deformación.
9. Una orden con varias unidades y varios equipos conserva correctamente la agrupación de datos y evidencias.

10. Los campos opcionales vacíos usan textos legibles y nunca muestran valores técnicos.
11. Un técnico ajeno a la orden no puede consultar ni descargar el reporte.
12. El archivo se almacena de forma privada y la URL de descarga expira.
13. Un fallo de generación es visible y permite reintento sin duplicar la versión vigente.
14. Después de una corrección se genera una versión nueva y la anterior se conserva para auditoría.
15. Una prueba de regresión visual compara portada, detalle y páginas fotográficas contra documentos de referencia aprobados.

12. Casos mínimos de prueba

- Una unidad, un equipo y cinco fotografías verticales.
- Varias unidades y varios equipos por unidad.
- Fotografías verticales, horizontales y con orientación EXIF.
- Varias fotografías dentro de la misma categoría.
- Equipo con campos opcionales vacíos.
- Equipo con discrepancia y valores observados.
- Observaciones generales y de checklist de varias líneas.
- Imagen dañada o faltante.
- Reintento después de un fallo temporal de almacenamiento.
- Corrección y generación de una segunda versión.
- Intento de descarga por administrador, técnico asignado y técnico ajeno.

13. Definiciones pendientes de negocio y diseño

Antes de aprobar la plantilla final se deben confirmar:

1. Archivo oficial del logotipo, variantes permitidas y colores de marca.
2. Texto exacto de la leyenda de conformidad junto a la firma.
3. Si la fecha principal de portada será la programada, la de inicio o la de conclusión.
4. Si `serialNumber` siempre representa el VIN o si se requiere un campo separado.
5. Nombre comercial de cada dato proveniente de `rawSnapshot` que también deba mostrarse.
6. Política de retención y visibilidad de versiones anteriores.
7. Si el cliente tendrá posteriormente un canal directo de descarga o envío.
8. Si se requiere simultáneamente una variante A4 además del formato Carta.

Documentación técnica

Proyecto: PUK — Sistema de órdenes de servicio de Bull Dog Solution

Última actualización: 2026-09-02

Alcance: estado implementado en la rama actual

Este directorio describe la arquitectura y operación técnica del sistema. A diferencia de `SPECS.md` y `docs/ERS.md`, que también contienen requisitos y decisiones de producto, estos documentos reflejan los componentes que existen actualmente en el repositorio.

Contenido

1. Arquitectura: contexto, contenedores, componentes, dependencias y decisiones estructurales.
2. Modelo de datos: diagrama entidad-relación, agregados, restricciones y estados.
3. Backend y flujos: módulos de API, workers y secuencias críticas.
4. API HTTP: convenciones, autenticación y catálogo de endpoints.
5. Frontend: estructura Vue, estado, rutas y flujo de datos.
6. Infraestructura y operación: Docker, configuración, despliegue, observabilidad y comandos.
7. Seguridad y calidad: controles implementados y estrategia de pruebas.

Fuentes de verdad

Tema	Fuente principal
Persistencia	`prisma/schema.prisma` y `prisma/migrations/`
Contratos de entrada	`packages/contracts/src/`
Rutas HTTP	Routers de `apps/api/src/modules/`
Documento OpenAPI	`apps/api/src/docs/openapi.ts` y `/docs`
Composición de servicios	`apps/api/src/server.ts` y `apps/api/src/app.ts`
Rutas de interfaz	`apps/web/src/router/` y módulos web
Infraestructura local	`docker-compose.yml` e `infra/docker/`
Reglas funcionales	`SPECS.md`, `docs/ERS.md` y servicios de dominio

Cuando la documentación y el comportamiento diverjan, el esquema, las migraciones, los contratos y el código ejecutable tienen precedencia. La documentación debe actualizarse en el mismo cambio que modifique esas

superficies.

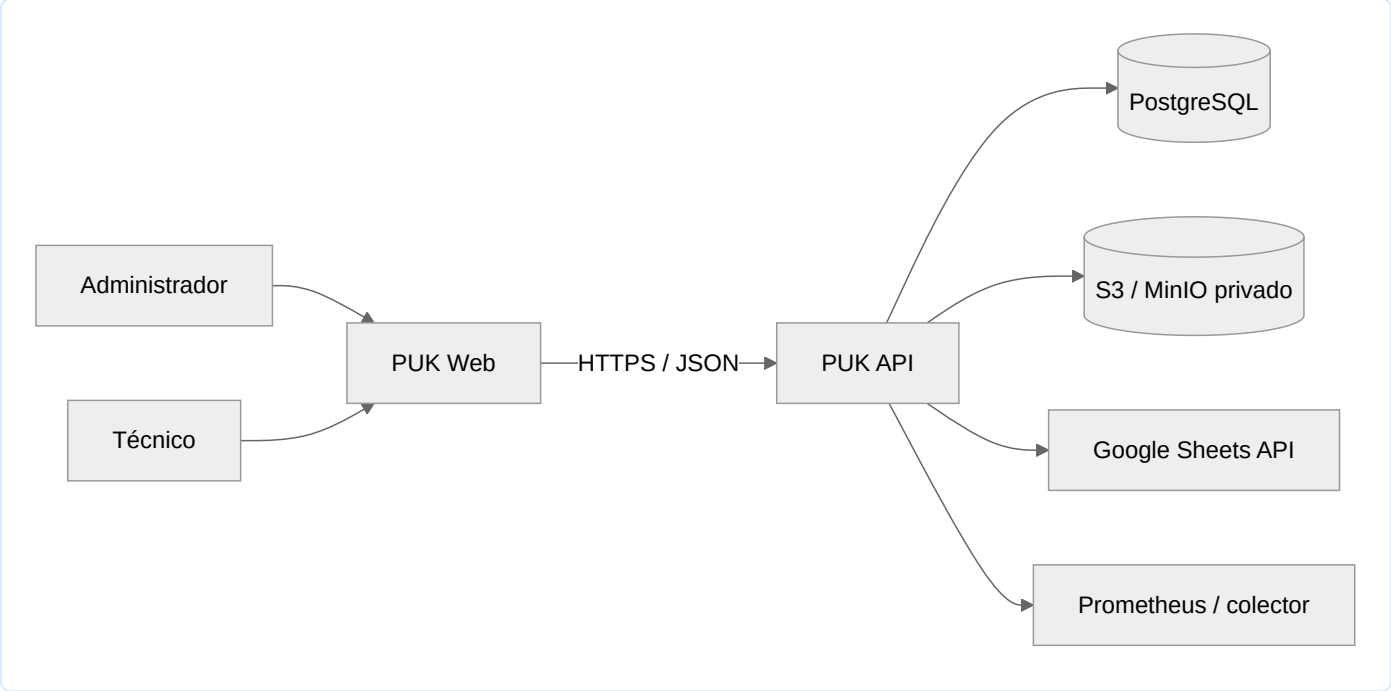
Arquitectura

Resumen

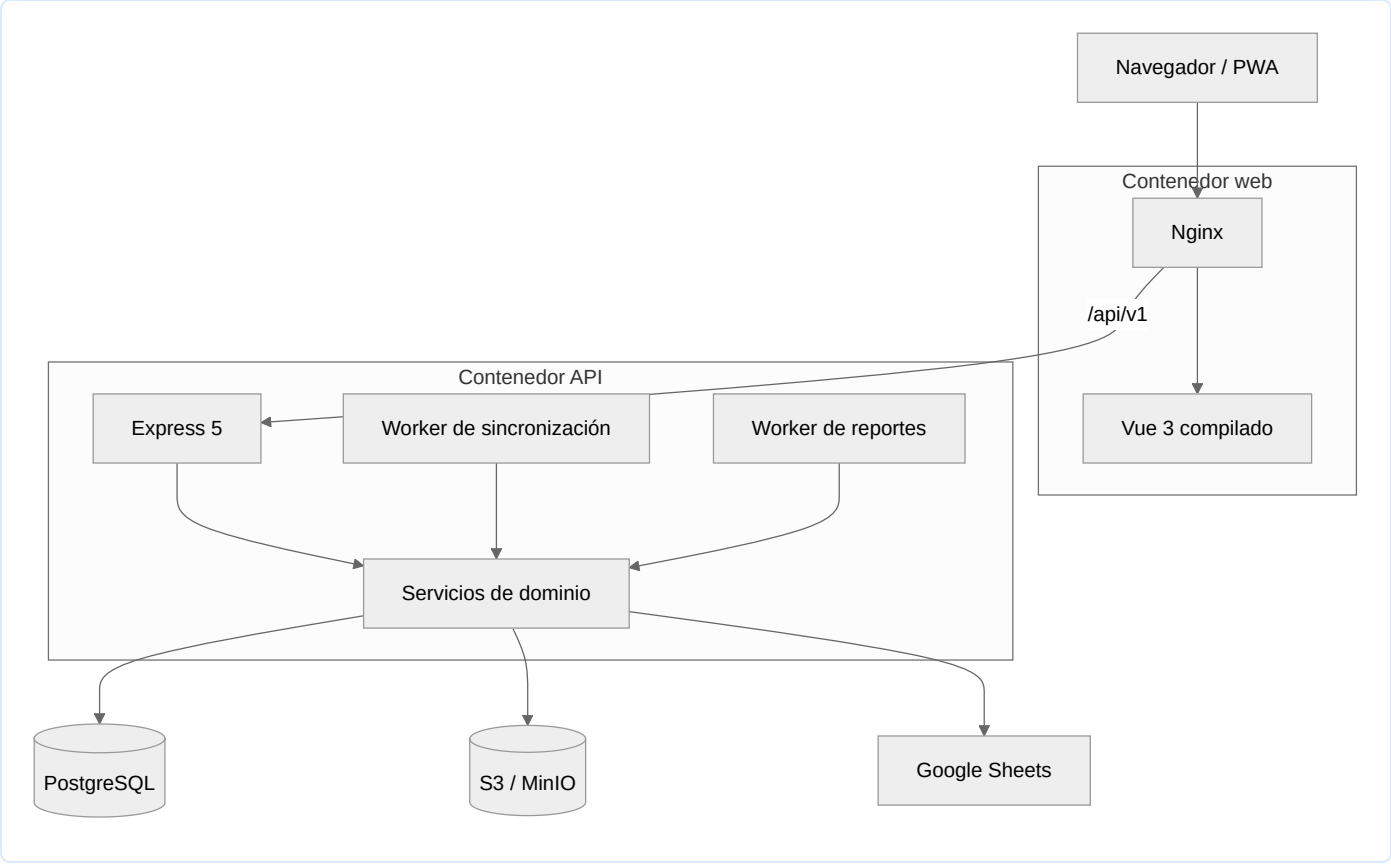
PUK es un monorepo TypeScript con un frontend Vue y una API Express desplegables por separado. La API implementa un monolito modular: autenticación, órdenes, ejecución técnica, catálogos, sincronización, archivos y reportes comparten proceso y base de datos, pero están separados por módulos de dominio.

PostgreSQL conserva el estado transaccional. Google Sheets sigue siendo la fuente externa de clientes y equipos. Las fotografías, firmas y PDFs viven en almacenamiento compatible con S3; PostgreSQL guarda únicamente metadatos y claves de objeto.

Diagrama de contexto

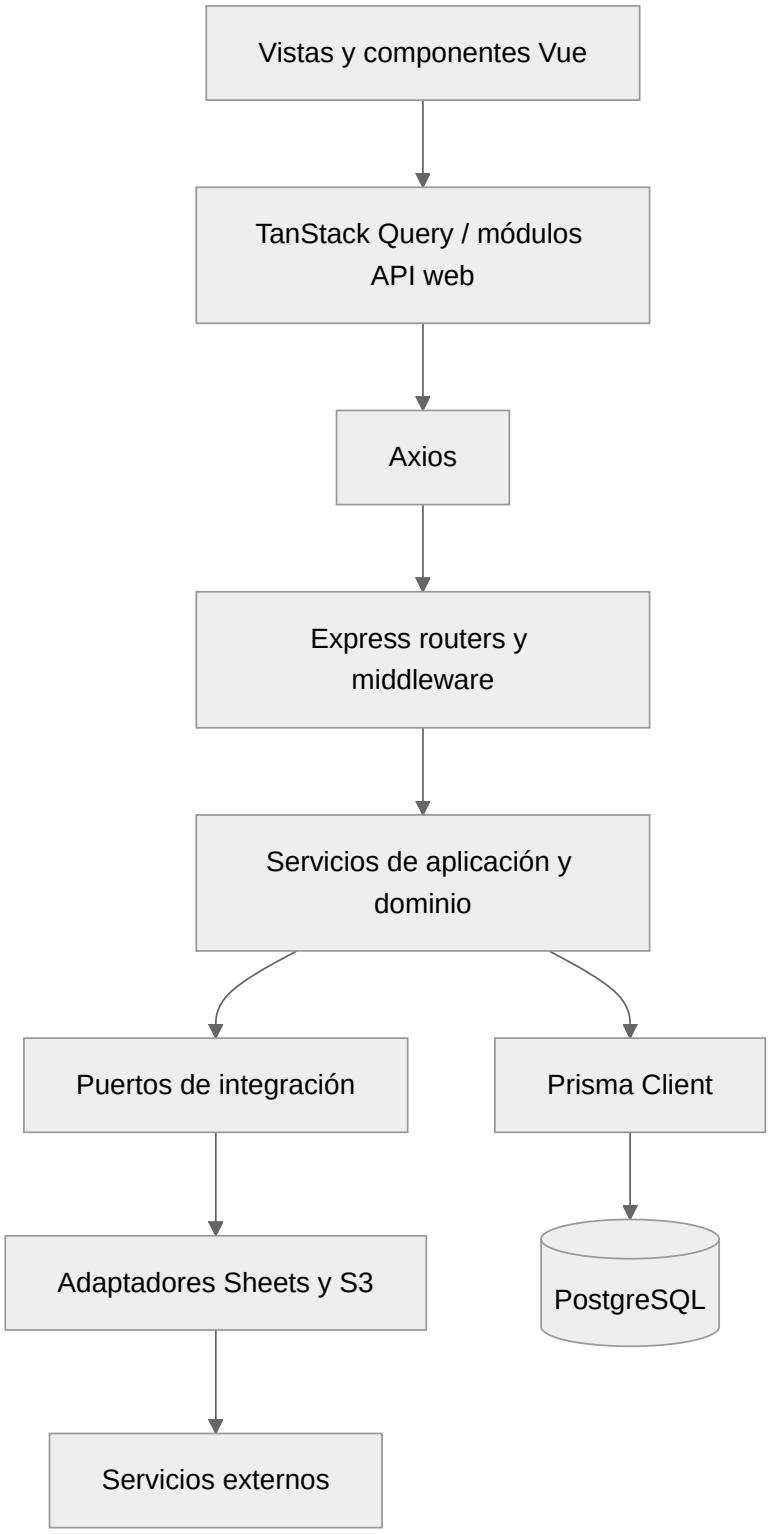


Contenedores desplegables



En Docker Compose, Nginx sirve el SPA y envía `/api` al contenedor de API. PostgreSQL y MinIO usan volúmenes persistentes. `minio-init` crea el bucket y bloquea el acceso anónimo.

Capas lógicas



Responsabilidades

- El frontend coordina navegación, formularios, cámara, firma, estados de carga y consumo HTTP.
- Los routers autentican, autorizan, validan y traducen HTTP; no deben contener reglas de negocio.
- Los servicios validan estados, propiedad, concurrencia y efectos del dominio.
- Prisma implementa persistencia y transacciones.
- Los puertos `SheetsPort` y `ObjectStoragePort` aíslan SDKs y permiten adaptadores simulados.
- `packages/contracts` comparte esquemas Valibot y tipos de entrada entre API y web.

Estructura del monorepo

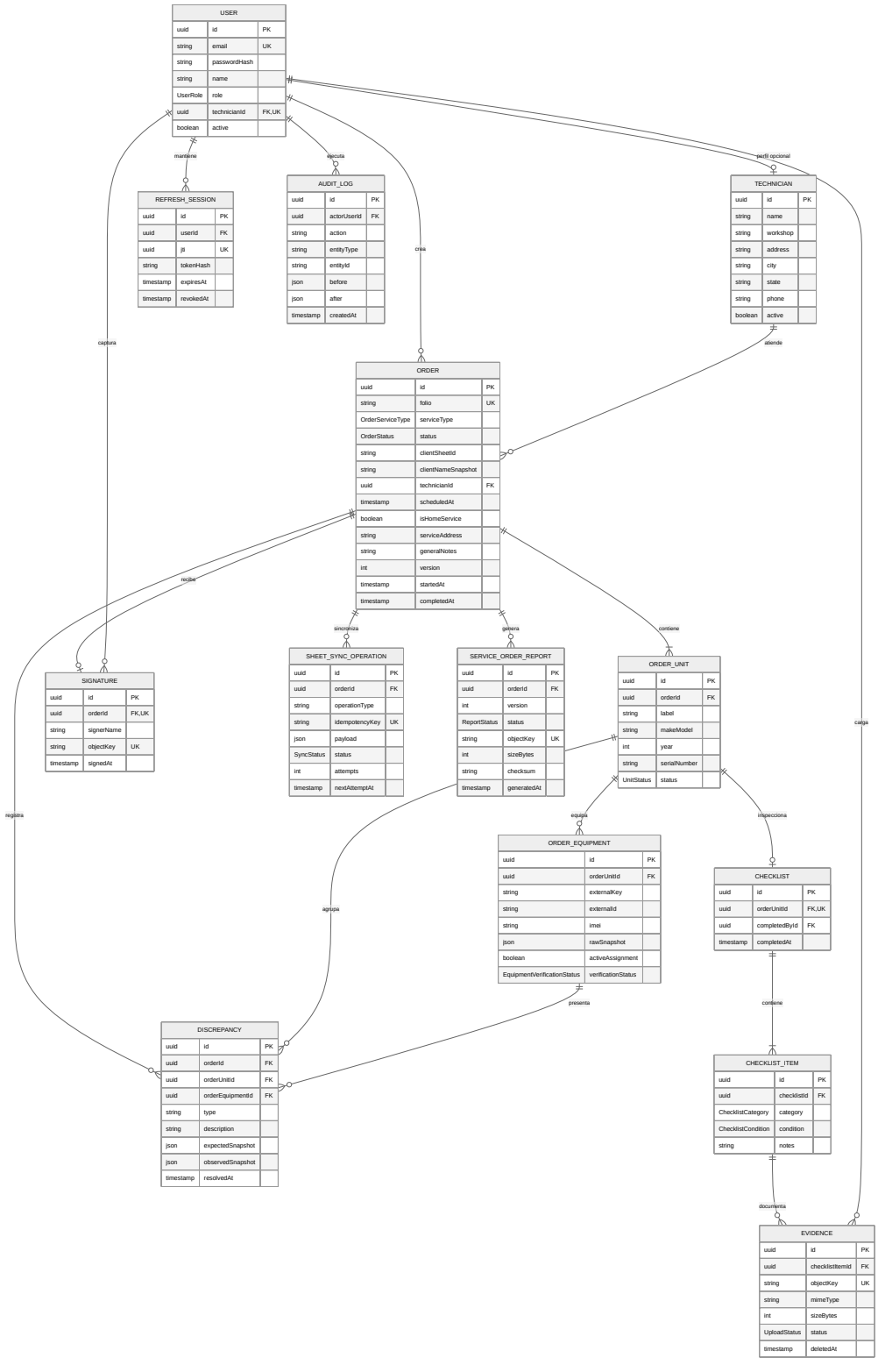
```
apps/
├── api/                API, workers, integraciones y OpenAPI
└── web/                SPA/PWA Vue
packages/
├── contracts/          DTOs y validaciones compartidas
├── eslint-config/      reglas de lint
└── typescript-config/  configuraciones TS compartidas
prisma/                 esquema, migraciones y seed
infra/docker/           imágenes de API/web y Nginx
docs/                   requisitos, decisiones y documentación técnica
```

Decisiones arquitectónicas vigentes

- Monolito modular antes que microservicios.
- Catálogos externos con snapshots locales dentro de la orden.
- Escrituras a Sheets mediante outbox persistente y reintentos.
- Carga directa de evidencias a S3 usando URLs prefirmadas.
- Reportes PDF generados en segundo plano y versionados.
- JWT de acceso en memoria del navegador; refresh token en cookie HTTP-only.
- TanStack Query para estado remoto; Pinia se limita a sesión y estado global de UI.
- Mocks de Sheets, almacenamiento y claves JWT únicamente en desarrollo con valores `development-only`.

Modelo de datos

Diagrama entidad-relación


USER
uuid id PK
string email UK
string passwordHash
string name
UserRole role
uuid technicianId FK,UK
boolean active
mantiene
ejecuta
perfil opcional
REFRESH_SESSION
uuid id PK
uuid userId FK
uuid jti UK
string tokenHash
timestamp expiresAt
timestamp revokedAt
AUDIT_LOG
uuid id PK
uuid actorUserId FK
string action
string entityType
string entityId
json before
json after
timestamp createdAt
crea
TECHNICIAN
uuid id PK
string name
string workshop
string address
string city
string state
string phone
boolean active
atiende
captura
ORDER
uuid id PK
string folio UK
OrderServiceType serviceType
OrderStatus status
string clientSheetId
string clientNameSnapshot
uuid technicianId FK
timestamp scheduledAt
boolean isHomeService
string serviceAddress
string generalNotes
int version
timestamp startedAt
timestamp completedAt
recibe
sincroniza
genera
contiene
carga
SIGNATURE
uuid id PK
uuid orderId FK,UK
string signerName
string objectKey UK
timestamp signedAt
SHEET_SYNC_OPERATION
uuid id PK
uuid orderId FK
string operationType
string idempotencyKey UK
json payload
SyncStatus status
int attempts
timestamp nextAttemptAt
SERVICE_ORDER_REPORT
uuid id PK
uuid orderId FK
int version
ReportStatus status
string objectKey UK
int sizeBytes
string checksum
timestamp generatedAt
ORDER_UNIT
uuid id PK
uuid orderId FK
string label
string makeModel
int year
string serialNumber
UnitStatus status
registra
equipa
inspecciona
agrupa
ORDER_EQUIPMENT
uuid id PK
uuid orderUnitId FK
string externalKey
string externalId
string imei
json rawSnapshot
boolean activeAssignment
EquipmentVerificationStatus verificationStatus
presenta
CHECKLIST
uuid id PK
uuid orderUnitId FK,UK
uuid completedById FK
timestamp completedAt
contiene
DISCREPANCY
uuid id PK
uuid orderId FK
uuid orderUnitId FK
uuid orderEquipmentId FK
string type
string description
json expectedSnapshot
json observedSnapshot
timestamp resolvedAt
CHECKLIST_ITEM
uuid id PK
uuid checklistId FK
ChecklistCategory category
ChecklistCondition condition
string notes
documenta
EVIDENCE
uuid id PK
uuid checklistItemId FK
string objectKey UK
string mimeType
int sizeBytes
UploadStatus status
timestamp deletedAt

Agregado principal

`Order` es la raíz del agregado operativo. Conserva el cliente como snapshot, pertenece opcionalmente a un técnico y contiene unidades. Cada unidad agrupa equipos, checklist, fotografías y discrepancias. La firma se captura una vez por orden. Sincronizaciones y reportes son efectos versionados relacionados con la orden.

Enumeraciones de estado

Entidad	Estados
Orden	`DRAFT`, `ASSIGNED`, `IN_PROGRESS`, `REQUIRES_CORRECTION`, `COMPLETED`, `FAILED_APPOINTMENT`, `CANCELLED`
Unidad	`PENDING`, `IN_PROGRESS`, `COMPLETED`
Verificación	`PENDING`, `MATCHED`, `DISCREPANCY_REPORTED`
Evidencia	`PENDING`, `COMPLETED`, `REJECTED`, `DELETED`
Sincronización	`PENDING`, `PROCESSING`, `SYNCED`, `FAILED`
Reporte	`PENDING`, `PROCESSING`, `READY`, `FAILED`

Restricciones relevantes

- `Order.folio` es único.
- Un usuario solo puede vincularse con un técnico.
- El número de serie de una unidad es único dentro de su orden.
- Solo existe un checklist y una firma por unidad/orden respectivamente.
- Cada categoría aparece una sola vez en cada checklist.
- Las claves de evidencia, firma y reporte son únicas en almacenamiento.
- El par `(orderId, version)` identifica una versión de reporte.
- `idempotencyKey` impide duplicar operaciones de sincronización.
- Una restricción parcial de migración impide asignar simultáneamente el mismo `externalKey` a más de una orden activa.
- `Order.version` implementa concurrencia optimista para edición administrativa.

Eliminación y retención

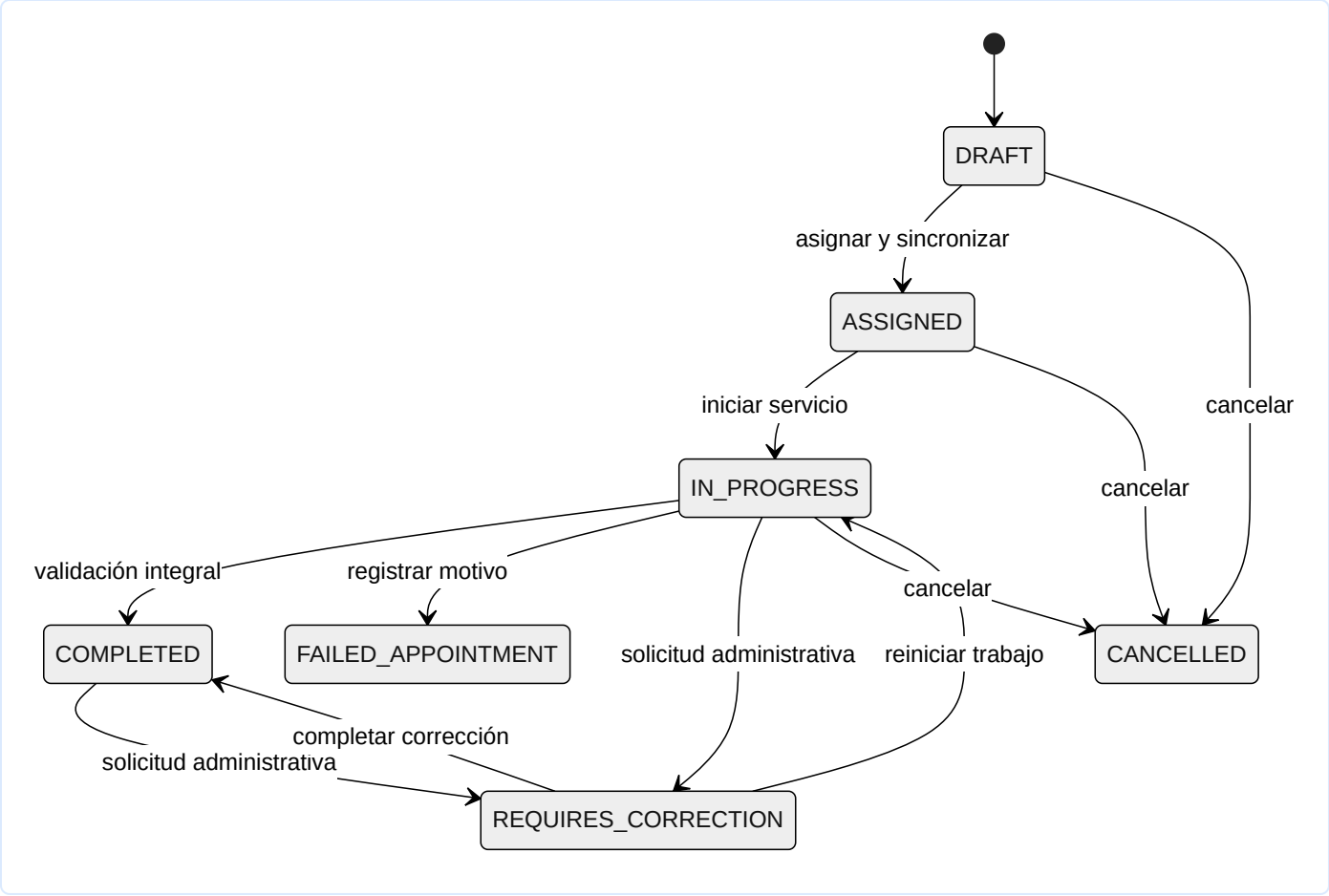
Unidades, checklist, items, evidencias, firmas y reportes usan cascada al eliminar su padre. Usuarios y técnicos usan restricciones o `SET NULL` para conservar trazabilidad. La aplicación normalmente cancela o elimina lógicamente evidencias; no expone eliminación física de órdenes como operación habitual.

Backend y flujos

Módulos de la API

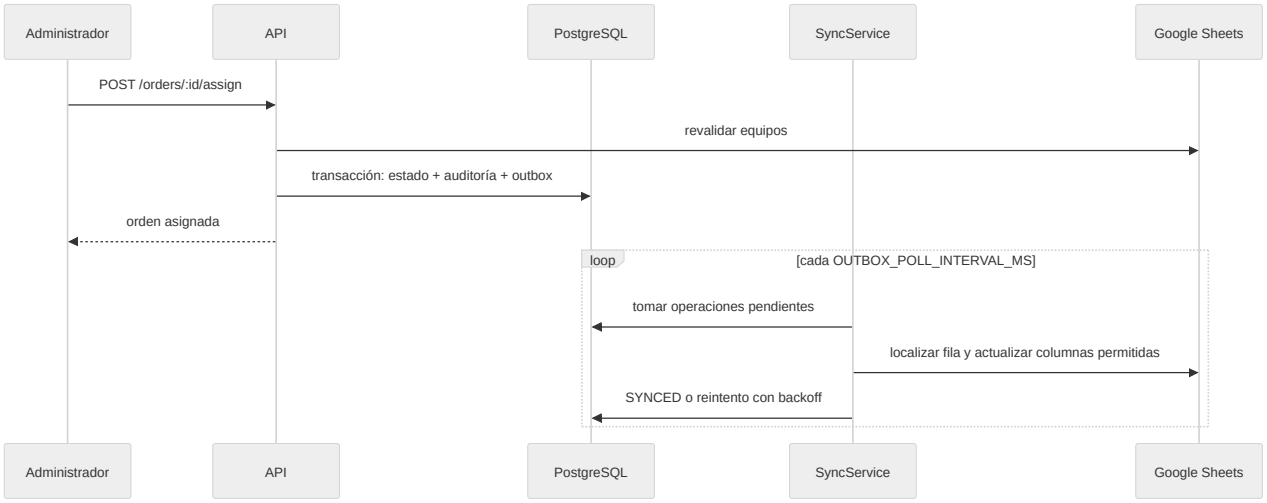
Módulo	Responsabilidad
`auth`	login, refresh, revocación y consulta de sesión
`users` / `technicians`	administración local y auditoría
`catalogs`	consulta y caché de clientes/equipos de Sheets
`orders`	borradores, unidades, equipos, asignación, cancelación y outbox
`execution`	inicio, verificación, discrepancias, checklist, evidencias, firma y cierre
`reports`	generación, versionado y descarga privada del PDF
`health`	liveness y readiness
`observability`	logs estructurados y métricas

Ciclo de una orden



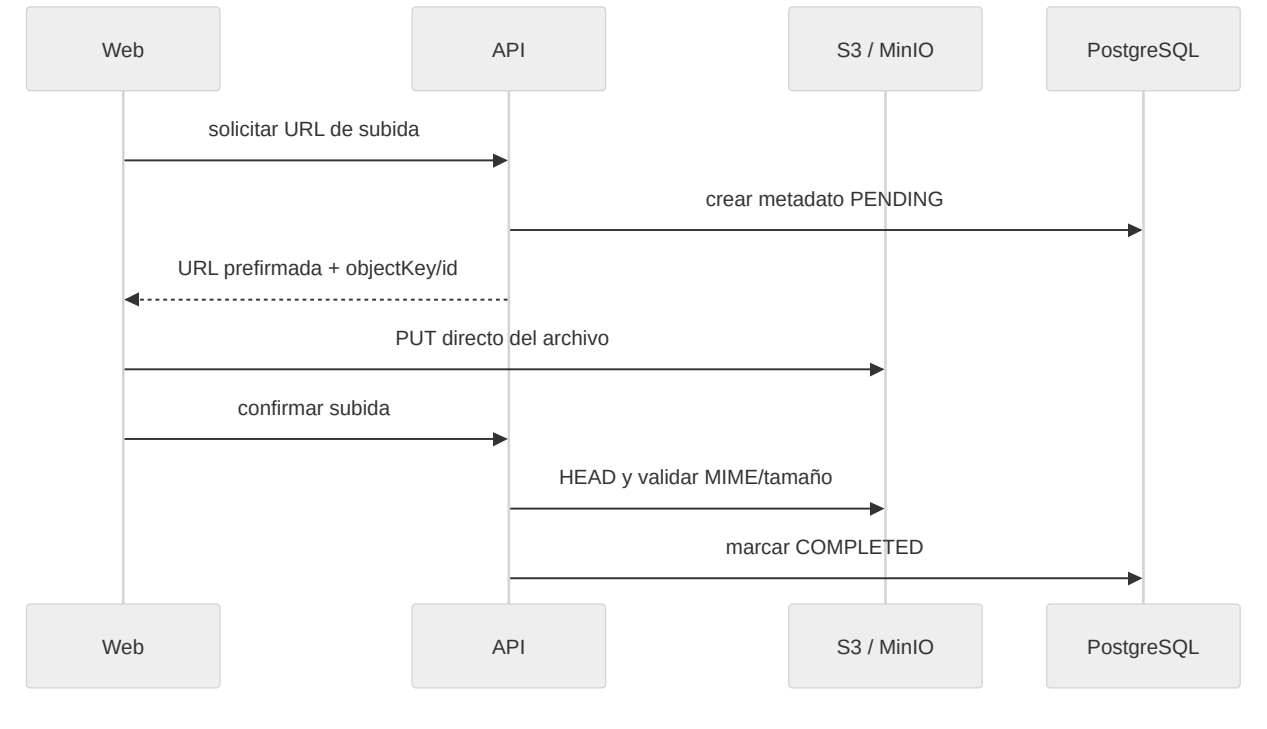
Para completar normalmente se validan equipos, cinco categorías de checklist, un mínimo de tres fotografías por unidad y firma con nombre. Una discrepancia documentada no bloquea el cierre. La cita fallida omite checklist, fotos y firma.

Asignación y sincronización con Sheets



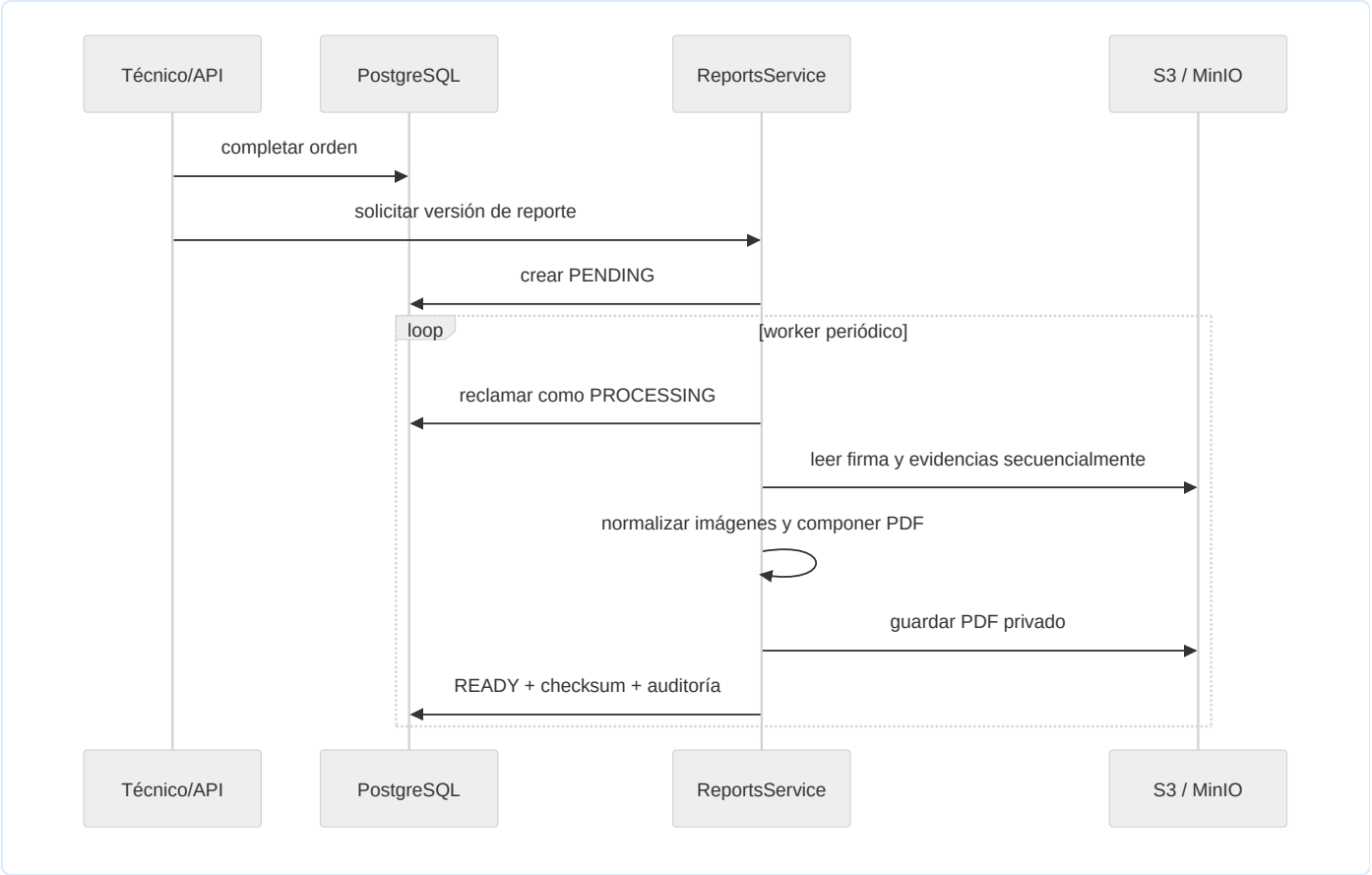
Las operaciones fallidas almacenan intentos, código/mensaje y siguiente fecha. El administrador puede devolverlas a `PENDING`. Google Sheets no se consulta para reconstruir órdenes cerradas: los equipos conservan snapshots.

Carga de fotografías y firma



Las claves siguen los prefijos `orders/{orderId}/units/...`, `orders/{orderId}/signature/...` y `orders/{orderId}/reports/...`. La base nunca almacena bytes ni Base64.

Generación de reporte PDF



PDFKit compone páginas Carta; Sharp corrige orientación y normaliza imágenes. Una evidencia por página ocupa el área útil sin mostrar la categoría del checklist. Un proceso `PROCESSING` abandonado por más de diez minutos vuelve a `PENDING`. Los errores públicos se sanitizan y un reintento no duplica la versión vigente.

Transacciones y auditoría

`domainTransaction` reúne mutaciones del dominio, auditorías y registros de outbox en una transacción Prisma. Acciones como creación, asignación, cierre, captura de evidencia, generación y descarga de reporte dejan eventos identificables. Los controladores no escriben directamente salvo consultas administrativas puntuales.

API HTTP

Convenciones

- Base autenticada: `/api/v1`.
- Cuerpo: JSON, excepto las cargas directas a S3.
- Validación: esquemas Valibot de `@puk/contracts`.
- Éxito: `{ "data": ..., "meta": { "requestId": ... } }`.
- Error: `{ "error": { "code": ..., "message": ..., "details"?: ... }, "meta": { "requestId": ... } }`.
- Fechas: ISO 8601 en UTC por HTTP; la presentación local pertenece al cliente/reporte.
- Documentación interactiva: `/docs`.

Autenticación y autorización

El access token se envía como `Authorization: Bearer <token>`. El refresh token se guarda en la cookie HTTP-only `puk_refresh`. Los roles son `ADMIN` y `TECHNICIAN`; además de capacidades, los servicios verifican que el técnico sea propietario de la orden.

Endpoints

Salud y observabilidad

Método	Ruta	Acceso	Uso
GET	`/health/live`	Público	proceso disponible
GET	`/health/ready`	Público	conexión a PostgreSQL
GET	`/metrics`	Infraestructura	formato Prometheus
GET	`/health/metrics`	Infraestructura	snapshot JSON

Sesión

Método	Ruta	Acceso	Uso
POST	`/api/v1/auth/login`	Público limitado	iniciar sesión
POST	`/api/v1/auth/refresh`	Cookie	rotar tokens
POST	`/api/v1/auth/logout`	Cookie	cerrar sesión actual

Método	Ruta	Acceso	Uso
POST	`/api/v1/auth/logout-all`	Autenticado	revocar todas las sesiones
GET	`/api/v1/auth/me`	Autenticado	usuario actual

Administración y catálogos

Método	Ruta	Acceso
GET/POST	`/api/v1/users`	Administrador
PATCH	`/api/v1/users/:id`	Administrador
GET/POST	`/api/v1/technicians`	Administrador
PATCH	`/api/v1/technicians/:id`	Administrador
GET	`/api/v1/external/clients`	Autenticado
GET	`/api/v1/external/equipment`	Autenticado
GET	`/api/v1/external/equipment/:externalKey`	Autenticado
POST	`/api/v1/external/refresh`	Administrador

Órdenes, unidades y equipos

Método	Ruta	Acceso / propósito
GET/POST	`/api/v1/orders`	listar propias/todas; crear como administrador
GET/PATCH	`/api/v1/orders/:id`	consultar; editar como administrador
POST	`/api/v1/orders/:id/assign`	asignar y encolar sincronización
POST	`/api/v1/orders/:id/cancel`	cancelar
GET	`/api/v1/orders/:id/audit`	auditoría administrativa
POST	`/api/v1/orders/:id/units`	agregar unidad
PATCH/DELETE	`/api/v1/orders/:id/units/:unitId`	modificar/eliminar unidad
POST	`/api/v1/orders/:id/units/:unitId/equipment`	asignar equipo
DELETE	`/api/v1/orders/:id/units/:unitId/equipment/:equipmentId`	retirar equipo
GET	`/api/v1/orders/:id/sync`	estado de sincronización

Método	Ruta	Acceso / propósito
POST	`/api/v1/orders/:id/sync/retry`	reintento administrativo

Ejecución técnica

Método	Ruta	Propósito
POST	`/api/v1/orders/:id/start`	iniciar trabajo
POST	`/api/v1/orders/:id/units/:unitId/equipment/:equipmentId/verify`	verificar equipo
POST	`/api/v1/orders/:id/units/:unitId/equipment/:equipmentId/discrepancy`	reportar diferencia
PUT	`/api/v1/orders/:id/units/:unitId/checklist`	guardar checklist
POST	`/api/v1/orders/:id/units/:unitId/checklist/:category/evidence/presign`	preparar foto
POST	`/api/v1/evidence/:id/complete`	confirmar foto
DELETE	`/api/v1/evidence/:id`	eliminar lógicamente foto
GET	`/api/v1/evidence/:id/url`	URL temporal de foto
POST	`/api/v1/orders/:id/signature/presign`	preparar firma
POST	`/api/v1/orders/:id/signature/complete?objectKey=...`	confirmar firma
GET	`/api/v1/orders/:id/signature/url`	URL temporal de firma
GET	`/api/v1/orders/:id/completeness`	requisitos pendientes
POST	`/api/v1/orders/:id/complete`	completar orden
POST	`/api/v1/orders/:id/fail-appointment`	registrar cita fallida
POST	`/api/v1/orders/:id/require-correction`	solicitar corrección

Reportes

Método	Ruta	Acceso / propósito
GET	`/api/v1/orders/:id/report`	metadatos de versión vigente
GET	`/api/v1/orders/:id/reports`	historial de versiones
POST	`/api/v1/orders/:id/report/generate`	generar/reintentar como administrador
GET	`/api/v1/orders/:id/report/download`	URL privada temporal

Contrato y compatibilidad

Los DTOs de escritura deben añadirse primero a `packages/contracts`. Los cambios incompatibles requieren una nueva versión de ruta o una migración coordinada de web y API. OpenAPI se genera/valida con `pnpm docs:build` y `pnpm docs:validate`; actualmente documenta la superficie principal, pero los routers son la lista exhaustiva.

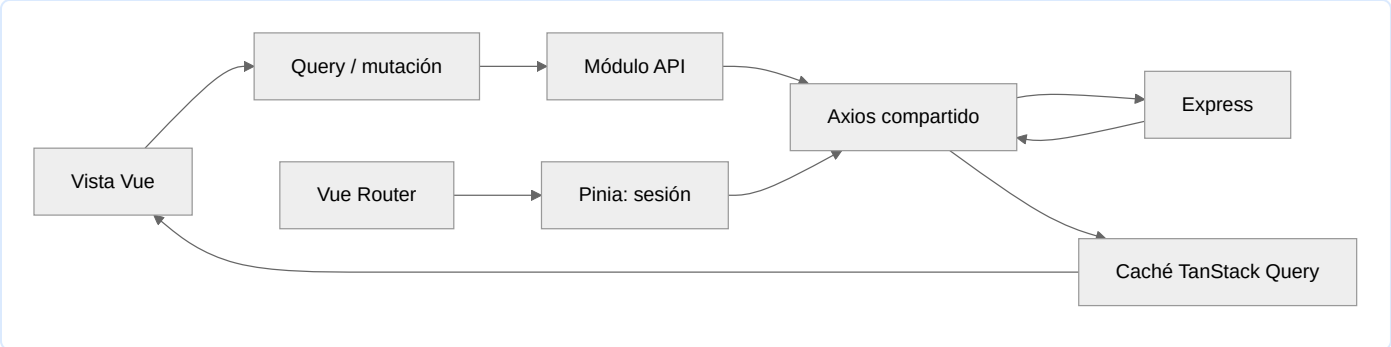
Frontend

Tecnologías y organización

La aplicación web usa Vue 3, Composition API, Vite, Vue Router, TanStack Query, Pinia, Axios, Naive UI y Tailwind CSS. Las rutas se cargan de forma diferida por módulo.

```
apps/web/src/
├── app/                    tema global
├── modules/
│   ├── auth/               inicio de sesión
│   ├── home/               tablero inicial
│   ├── external-catalogs/selectores de Sheets
│   ├── orders/             lista, alta y detalle administrativo
│   ├── service-execution/flujo móvil, cámara y firma
│   └── technicians/        directorio administrativo
├── router/                 composición y guardas
├── shared/                 HTTP y layout
└── stores/                 sesión
```

Flujo de datos



- TanStack Query mantiene datos remotos, caché, polling e invalidación.
- Pinia conserva usuario, access token en memoria e inicialización de sesión.
- Axios adjunta el access token y coordina un único refresh ante respuestas `401`.
- Formularios y modales usan estado local del componente.
- Tras mutaciones se invalidan órdenes, auditoría, sincronización o reporte según corresponda.

Rutas

Ruta	Vista	Perfil
`/login`	inicio de sesión	público
`/`	dashboard / órdenes recientes	autenticado
`/orders`	listado y filtros	autenticado; datos filtrados en API
`/orders/new`	asistente de creación	administrador
`/orders/:id`	detalle, evidencias, reporte y auditoría	autorizado sobre la orden
`/orders/:id/execute`	ejecución mobile-first	técnico asignado o administrador
`/technicians`	administración de técnicos	administrador

La guarda de navegación inicializa la sesión, redirige al login y aplica `requiresRole`. La autorización real siempre se repite en backend.

Evidencias y reportes

El navegador comprime fotografías a JPEG antes de solicitar/subir una URL prefirmada. La firma se captura con canvas y se sube como PNG. El detalle de la orden consulta el estado del reporte cada tres segundos mientras está pendiente o procesándose y obtiene una URL temporal cuando queda listo.

PWA

El proyecto incluye manifest e iconos instalables. Solo se cachean assets estáticos; no se promete operación offline ni se cachean respuestas autenticadas.

Infraestructura y operación

Servicios locales

Servicio	Imagen/proceso	Puerto predeterminado	Persistencia
`web`	Nginx + Vue compilado	4173 externo / 80 interno	no
`api`	Node.js 24 + Express	3000	no
`postgres`	PostgreSQL 18	5432	volumen `postgres-data`
`minio`	MinIO	9000 API, 9001 consola	volumen `minio-data`
`minio-init`	cliente `mc`	—	crea bucket privado

Los Dockerfiles son multi-stage. La API se despliega con dependencias de producción y usuario `node`; el frontend se construye con Vite y se sirve desde Nginx.

Inicio rápido

```
pnpm install
cp .env.example .env
docker compose up --build -d
pnpm exec prisma migrate deploy
pnpm prisma:seed
```

Para desarrollo con hot reload se pueden levantar PostgreSQL/MinIO con Docker y ejecutar `pnpm dev` desde la raíz. Las versiones admitidas son Node.js 24–26 y pnpm 11.

Variables de entorno

Grupo	Variables principales
HTTP	`PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL`
JWT	claves pública/privada, `JWT_ISSUER`, `JWT_AUDIENCE`, TTLs
Cookies	`COOKIE_DOMAIN`, `COOKIE_SECURE`, `COOKIE_SAME_SITE`
S3	endpoint, región, bucket, credenciales, path style

Grupo	Variables principales
Sheets	spreadsheet, cuenta de servicio, clave privada y nombres de pestañas
Límites	tamaños de evidencia/firma, TTL de URLs, caché
Resiliencia	timeouts, reintentos, circuit breaker y polling
Seed	correos y contraseñas iniciales de desarrollo

La API valida todas las variables al arrancar. Los valores `development-only` activan adaptadores simulados fuera de producción; nunca deben usarse en producción.

Procesos periódicos

- `SyncService` consulta el outbox cada `OUTBOX_POLL_INTERVAL_MS`, procesa hasta 20 operaciones y aplica backoff exponencial con jitter.
- `ReportsService` procesa hasta dos PDFs por ciclo. Recupera trabajos interrumpidos después de diez minutos.
- Ambos intervalos usan `unref()` y se cancelan durante el apagado ordenado.

Observabilidad

- `GET /health/live`: proceso activo.
- `GET /health/ready`: PostgreSQL responde.
- `GET /metrics`: métricas Prometheus.
- `GET /health/metrics`: snapshot JSON.
- Winston produce logs JSON y Morgan registra HTTP mediante el logger central.
- Cada petición obtiene `requestId`; puede propagarse uno válido desde el cliente.

Operaciones habituales

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm prisma:validate
pnpm docs:validate
pnpm exec prisma migrate deploy
```

Las migraciones se ejecutan una sola vez antes de activar una versión. PostgreSQL y el bucket requieren respaldos independientes. Debe probarse restauración de base y conservarse alineada con la política de retención de evidencias.

Despliegue

El orden recomendado es: construir imágenes, respaldar, aplicar migraciones, desplegar API, validar readiness, desplegar web y ejecutar smoke tests. `/api` debe publicarse bajo el mismo dominio cuando sea posible para simplificar cookies y CORS. Producción necesita TLS, cookies `Secure`, secretos externos al repositorio y bloqueo público del bucket.

Seguridad y calidad

Controles implementados

Identidad y sesión

- Contraseñas con Argon2id.
- Access y refresh JWT firmados con pares de claves separados.
- Access token de corta duración y refresh token rotativo.
- Solo se almacena el hash del refresh token en PostgreSQL.
- Cookie HTTP-only configurable con `Secure`, `SameSite` y dominio.
- Revocación por sesión y cierre de todas las sesiones.

API

- Helmet, CORS con allowlist y límite JSON de 1 MiB.
- Rate limiting general y específico para autenticación.
- Validación Valibot de body, params y query.
- Capacidades por rol más políticas de propiedad de la orden.
- Mensajes de autenticación genéricos y manejador uniforme de errores.
- `requestId` para correlación.

Archivos

- Bucket privado y URLs prefirmadas con expiración.
- Carga directa para evitar transportar imágenes por Express.
- Validación posterior de existencia, MIME y tamaño.
- Nombres de objeto generados por servidor.
- Fotografías y firmas fuera de PostgreSQL.
- PDFs privados, checksum SHA-256 y errores de generación sanitizados.
- Las respuestas públicas de reporte no exponen `objectKey`.

Integraciones y datos

- Credenciales de Google y S3 solo en servidor.
- Puertos abstraen integraciones externas.
- Timeouts, reintentos y circuit breaker para Sheets.
- Outbox e idempotencia para efectos externos.
- Concurrencia optimista en órdenes y restricción de asignación activa.
- Auditoría de acciones críticas con snapshots antes/después cuando aplica.

Roles

Capacidad	Administrador	Técnico
Usuarios y técnicos	Sí	No
Crear/editar/asignar órdenes	Sí	No
Consultar todas las órdenes	Sí	No
Ejecutar orden propia	Sí según endpoints	Sí
Consultar catálogos	Sí	Sí
Reintentar sincronización/reporte	Sí	No
Descargar reporte	Sí	Solo de orden asignada

Las capacidades declaradas son `users:manage`, `technicians:manage`, `orders:manage`, `orders:execute`, `catalogs:read` y `sync:retry`.

Estrategia de pruebas

Nivel	Herramientas	Cobertura principal
Unitario	Vitest	tokens, contraseñas, validación, estados, generador PDF, resiliencia
Integración API	Supertest + Vitest	autenticación, catálogos, permisos, formato HTTP
Integración DB	Prisma + PostgreSQL de prueba	restricciones, concurrencia, unicidad
Contrato	OpenAPI + pruebas	health, respuestas y referencias
E2E	Playwright	flujos de administrador y técnico
Build estático	TypeScript/Vue TSC/Vite	compatibilidad estricta y bundles

Pipeline mínimo

1. Instalación con lockfile.
2. Formato y lint.
3. Typecheck.
4. Pruebas unitarias e integración.
5. Validación Prisma y OpenAPI.
6. Build de API y web.
7. Escaneo de dependencias e imágenes.
8. Migraciones, despliegue y smoke tests.

Aspectos que requieren política externa

- Rotación y custodia de claves JWT, Google y S3.
- Retención legal de fotografías, firmas, auditoría y reportes.
- Control de acceso a `/metrics` en el proxy productivo.
- Backups cifrados y pruebas periódicas de restauración.
- Aprobación del logotipo y leyenda de conformidad del PDF.
- Monitoreo y alertas sobre outbox fallido, generación de reportes y espacio del bucket.